



BITS Pilani

Pilani Campus

Deep Neural Networks

Module 2

Deep Forward Neural Network

Dr . S. Suresh Kumar

Deep Neural Network



Disclaimer and Acknowledgement



- The content for these slides has been obtained from books and various other source on the Internet
- I here by acknowledge all the contributors for their material and inputs.
- I have provided source information wherever necessary
- I have added and modified the content to suit the requirements of the course

Session Agenda



- Multi-class Classification
- Deep Feedforward Neural Networks
- Activation Functions
- Computational Graph
- Forward Propagation

WHAT IS MULTI-CLASS CLASSIFICATION?

DEFINITION

Multi-class classification assigns each input to one of K classes, where $K > 2$.

Key Characteristics:

- Classes: $y \in \{1, 2, 3, \dots, K\}$ (or $\{0, 1, 2, \dots, K - 1\}$)
- Each example belongs to **exactly one** class
- Output: Probability distribution over all K classes
- Probabilities sum to 1: $\sum_{k=1}^K P(y = k|\mathbf{x}) = 1$

Example:

- Input: Image of a handwritten digit
- Output: Probabilities for each of 10 classes (0, 1, 2, ..., 9)
- Prediction: Class with highest probability

EXAMPLES OF MULTI-CLASS CLASSIFICATION

1 Handwritten Digit Recognition (MNIST)

- Input: 28×28 pixel image
- Output: One of 10 digits (0-9)

2 Image Classification (ImageNet)

- Input: Color image
- Output: One of 1000 object categories (cat, dog, car, plane, ...)

3 Text Classification

- Input: News article text
- Output: Topic category (politics, sports, technology, entertainment, business)

4 Speech Recognition

- Input: Audio features
- Output: Phoneme class (multiple phonemes in a language)

MULTI-CLASS CLASSIFICATION: VISUAL EXAMPLE



Key insight: We need **multiple output neurons** (one per class) and a way to convert their outputs into a valid probability distribution.

Decision boundaries separate $K = 3$ classes in feature space.



THE FOUR COMPONENTS FOR MULTI-CLASS

A complete multi-class classification system consists of:

- ① **Data:** Input features and class labels $y \in \{1, 2, \dots, K\}$
- ② **Model:** Linear model with K output neurons (no hidden layers)
- ③ **Activation:** Softmax function to produce probability distribution
- ④ **Objective Function:** Categorical cross-entropy loss
- ⑤ **Learning Algorithm:** Mini-batch stochastic gradient descent

Note: We now use **mini-batch SGD** instead of single-example SGD for better efficiency and stability.

DATA REPRESENTATION

Input Features:

$$\mathbf{X} \in \mathbb{R}^{N \times (d+1)} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ 1 & x_1^{(2)} & \dots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \dots & x_d^{(N)} \end{bmatrix}$$

Class Labels: Two representations

1. Integer encoding: $\mathbf{y} \in \{1, 2, \dots, K\}^N$

$$\mathbf{y} = [3 \quad 1 \quad 2 \quad 1 \quad 3]^T$$

2. One-hot encoding: $\mathbf{Y} \in \{0, 1\}^{N \times K}$ (preferred for computation)

ONE-HOT ENCODING

Definition: Convert class label to binary vector of length K

For class k : All zeros except 1 at position k

Example with $K = 4$ classes:

Class 1: $\mathbf{y} = [1, 0, 0, 0]^T$

Class 2: $\mathbf{y} = [0, 1, 0, 0]^T$

Class 3: $\mathbf{y} = [0, 0, 1, 0]^T$

Class 4: $\mathbf{y} = [0, 0, 0, 1]^T$

Why one-hot?

- Treats all classes equally (no implicit ordering) Matches the format of softmax
- output probabilities Simplifies loss computation
- Standard in deep learning frameworks
-

WEIGHT MATRIX FOR MULTI-CLASS CLASSIFICATION

For K classes and d input features:

Weight Matrix: $\mathbf{W} \in \mathbb{R}^{(d+1) \times K}$

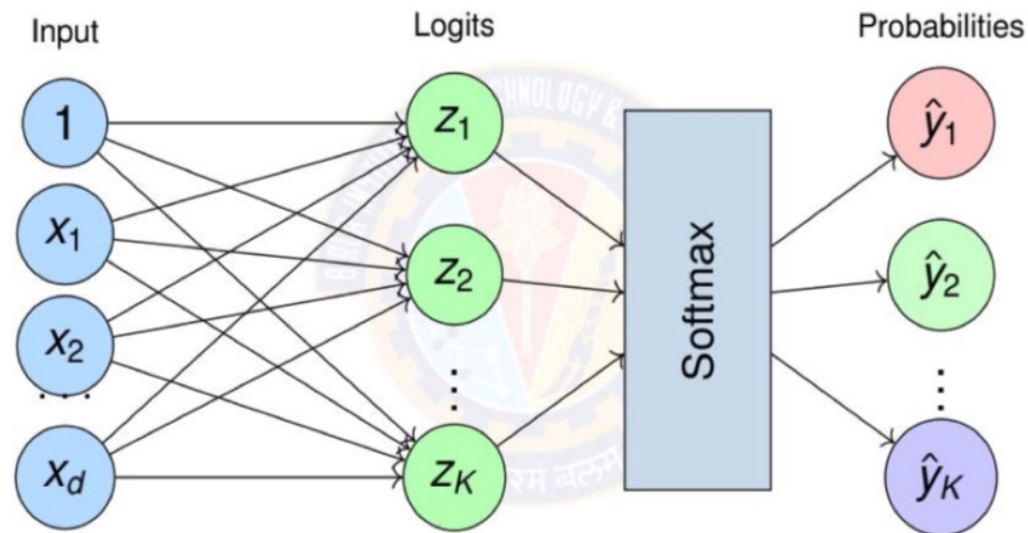
$$\mathbf{W} = \begin{bmatrix} w_{0,1} & w_{0,2} & \cdots & w_{0,K} \\ w_{1,1} & w_{1,2} & \cdots & w_{1,K} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \cdots & w_{d,K} \end{bmatrix}$$

- Each **column** $\mathbf{w}_k$ represents weights for class k
- Each class has its own linear model
- Row 0: Bias terms for all classes
- Total parameters: $(d + 1) \times K$

Compare to binary: 1 weight vector vs K weight vectors!

MULTI-CLASS MODEL ARCHITECTURE

Architecture: Multiple output neurons with softmax activation



$$\mathbf{z} = \mathbf{W}^T \mathbf{x} \text{ (linear)} \rightarrow \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \text{ (activation)}$$

LINEAR MODEL: COMPUTING LOGITS

Step 1: Compute logits (pre-activation values)

For a single input $\mathbf{x}$:

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} = \begin{bmatrix} \mathbf{w}_1^T \mathbf{x} \\ \mathbf{w}_2^T \mathbf{x} \\ \vdots \\ \mathbf{w}_K^T \mathbf{x} \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix} \quad (15)$$

where $z_k = \mathbf{w}_k^T \mathbf{x} = w_{0,k} + w_{1,k}x_1 + \dots + w_{d,k}x_d$

For entire dataset (batch):

$$\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{N \times K} \quad (16)$$

Each row of $\mathbf{Z}$ contains logits for one example.

Logits are **unbounded** values in $(-\infty, \infty) \rightarrow$ not probabilities yet!



THE SOFTMAX FUNCTION

Purpose: Convert K logits into a valid probability distribution

DEFINITION:

For logit vector $\mathbf{z} = [z_1, z_2, \dots, z_K]^T$

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad \text{for } k = 1, 2, \dots, K \quad (17)$$

Output: Probability vector $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \dots, \hat{y}_K]^T$

$$\hat{y}_k = P(y = k|\mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad (18)$$

Interpretation: $\hat{y}_k$ is the predicted probability for class k

PROPERTIES OF SOFTMAX

Key Properties:

- 1 Output range: $\hat{y}_k \in (0, 1)$ for all k
- 2 Probability distribution: $\sum_{k=1}^K \hat{y}_k = 1$
- 3 Preserves order: If $z_i > z_j$, then $\hat{y}_i > \hat{y}_j$
- 4 Translation invariant: $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c)$ for any constant c
- 5 Differentiable: Enables gradient-based learning
- 6 Reduces to sigmoid: When $K = 2$, softmax is equivalent to sigmoid

The exponential amplifies differences: larger logits get much larger probabilities!

SOFTMAX EXAMPLE: NUMERICAL COMPUTATION

Given logits: $\mathbf{z} = [2.0, 1.0, 0.1]^T$ for $K = 3$ classes

Step 1: Compute exponentials

$$e^{z_1} = e^{2.0} \approx 7.39$$

$$e^{z_2} = e^{1.0} \approx 2.72$$

$$e^{z_3} = e^{0.1} \approx 1.11$$

Step 2: Compute sum

$$\sum_{j=1}^3 e^{z_j} = 7.39 + 2.72 + 1.11 = 11.22$$

Step 3: Normalize

$$\hat{y}_1 = \frac{7.39}{11.22} \approx 0.659$$

$$\hat{y}_2 = \frac{2.72}{11.22} \approx 0.242$$

$$\hat{y}_3 = \frac{1.11}{11.22} \approx 0.099$$

Step 4: Predicted class:

$\arg \max_k \hat{y}_k = 1$ (highest probability)

COMPLETE FORWARD PASS

For a batch of N examples $\mathbf{X}$:

Step 1: Compute logits $\mathbf{Z} = \mathbf{XW} \in \mathbb{R}^{N \times K}$ (19)

Step 2: Apply softmax $\hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in [0, 1]^{N \times K}$ (20)

(softmax applied row-wise to each example)

Step 3: Predict class $\hat{k} = \arg \max_k \hat{y}_k$ (21)

Each row of $\hat{\mathbf{Y}}$ is a probability distribution over K classes.

CATEGORICAL CROSS-ENTROPY LOSS

Total loss over dataset:

$$J(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)}) \quad (22)$$

Goal: Minimize $J(\mathbf{W})$ to find optimal weights $\mathbf{W}^*$

UNDERSTANDING CATEGORICAL CROSS-ENTROPY

Intuition:

- Loss depends only on predicted probability for the **true class**
- Heavily penalizes **confident wrong predictions**
- Encourages high probability for correct class

Example: True class is 2 ($\mathbf{y} = [0, 1, 0]$)

Prediction	$\hat{y}_1$	$\hat{y}_2$	Loss = $-\log(\hat{y}_2)$
Good	0.1	0.8	0.22
Medium	0.3	0.5	0.69
Bad	0.7	0.2	1.61
Very bad	0.9	0.05	3.00

Loss increases exponentially as predicted probability for true class decreases!

WHY CATEGORICAL CROSS-ENTROPY?

Advantages:

- Probabilistic interpretation: Derived from maximum likelihood
- Appropriate for multi-class: Naturally handles K classes
- Works well with softmax: Numerically stable gradient
- Penalizes wrong confidence: Heavily penalizes confident mistakes
- Convex: For linear models (like ours)

Connection to binary cross-entropy:

When $K = 2$:

$$\text{Categorical: } - [y_1 \log(\hat{y}_1) + y_2 \log(\hat{y}_2)]$$

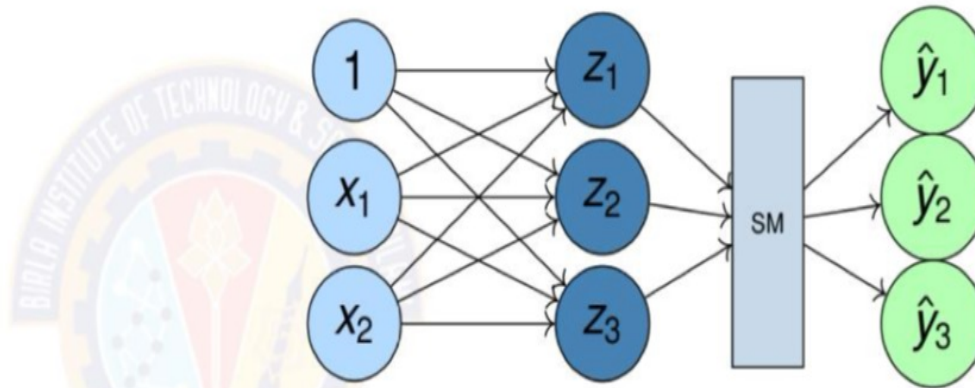
$$\text{Binary: } - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

They are equivalent! (with $y_2 = y$ and $y_1 = 1 - y$)

NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Classify 4 data points into 3 classes

x_1	x_2	Class
1	2	1
2	1	2
2	3	1
3	2	3



Parameters:

- $K = 3$ classes, $d = 2$ features, $N = 4$ examples
- Batch size: $B = 2$ (mini-batch)
- Learning rate: $\eta = 0.1$

NUMERICAL EXAMPLE: DATA REPRESENTATION

Design Matrix: (with bias column) $\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 2 \end{bmatrix} \in \mathbb{R}^{4 \times 3}$

One-hot Labels: $\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4 \times 3}$

(class 1)
(class 2)
(class 1)
(class 3)

Weight Matrix: Initialize randomly $\mathbf{W}^{(0)} = \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} \in \mathbb{R}^{3 \times 3}$

NUMERICAL EXAMPLE: FIRST MINI-BATCH (EXAMPLE)

Mini-batch: First 2 examples after shuffling

$$\mathbf{X}_{\mathcal{B}} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \mathbf{Y}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 1: Compute logits

$$\begin{aligned} \mathbf{Z}_{\mathcal{B}} &= \mathbf{X}_{\mathcal{B}} \mathbf{W}^{(0)} \\ &= \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} \\ &= \begin{bmatrix} 0.1 & 0.4 & 0.3 \\ 0.4 & 0.2 & 0.1 \end{bmatrix} \end{aligned}$$

NUMERICAL EXAMPLE: SOFTMAX COMPUTATION

Step 2: Apply softmax to each row

Softmax for example 1: $\hat{\mathbf{y}}^{(1)} \approx \begin{bmatrix} 0.280 \\ 0.378 \\ 0.342 \end{bmatrix}$

Similarly for example 2: $\hat{\mathbf{y}}^{(2)} \approx [0.387, 0.315, 0.298]^T$

For the entire batch: $\hat{\mathbf{Y}}_{\mathcal{B}} = \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix}$

NUMERICAL EXAMPLE: GRADIENT AND UPDATE

Step 3: Compute gradient

$$\begin{aligned}
 \nabla J &= \frac{1}{B} \mathbf{X}_B^T (\hat{\mathbf{Y}}_B - \mathbf{Y}_B) \\
 \hat{\mathbf{Y}}_B - \mathbf{Y}_B &= \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix} \\
 \nabla J &= \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix} \\
 &\approx \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix}
 \end{aligned}$$

NUMERICAL EXAMPLE: WEIGHT UPDATE

Step 4: Update weights

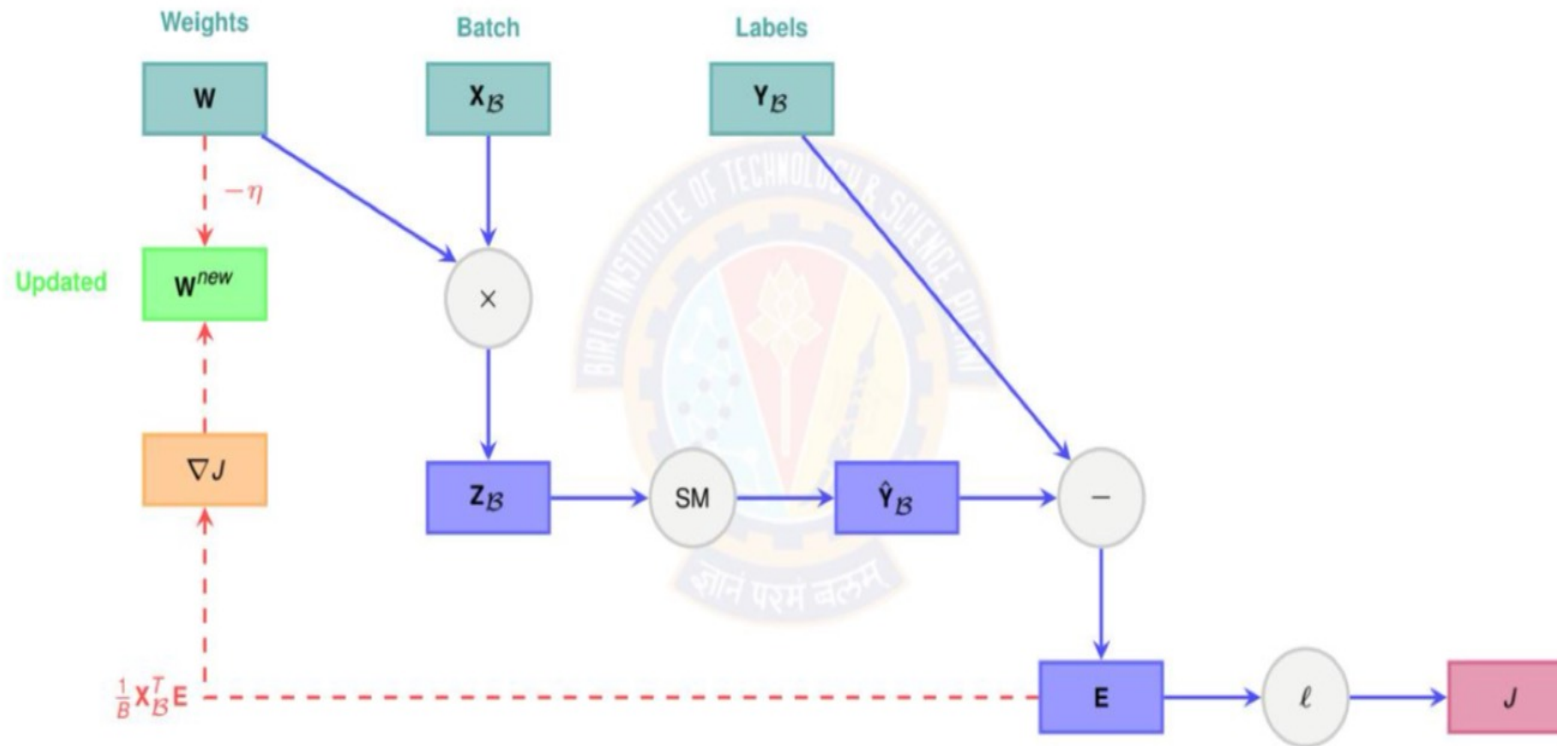
$$\begin{aligned}\mathbf{W}^{(1)} &= \mathbf{W}^{(0)} - \eta \nabla J \\ &= \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix} \\ &= \begin{bmatrix} 0.117 & -0.085 & 0.168 \\ 0.202 & 0.150 & -0.147 \\ -0.047 & 0.196 & 0.051 \end{bmatrix}\end{aligned}$$

Progress:

- Weights adjusted based on mini-batch gradient
- Process next mini-batch (examples 3-4) with updated weights
- Continue for multiple epochs until convergence

This completes one iteration of mini-batch SGD!

COMPUTATIONAL GRAPH FOR MINI-BATCH FORWARD



Mini-batch processes B examples simultaneously — efficient and stable!

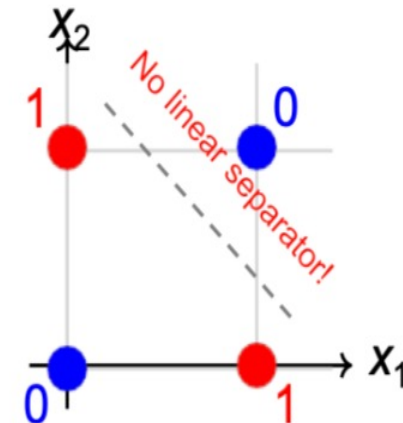
THE XOR PROBLEM

XOR (Exclusive OR): A classic non-linearly separable problem

Visual Representation:

Truth Table:

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0



Challenge: No single straight line can separate the two classes!

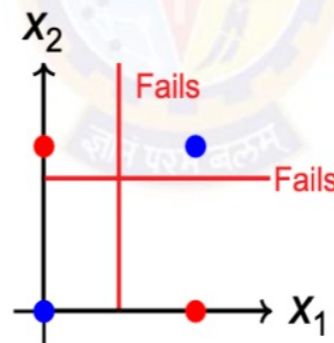
WHY SINGLE NEURON FAILS ON XOR

Single neuron model:

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + b) \quad (2)$$

where σ is any activation function.

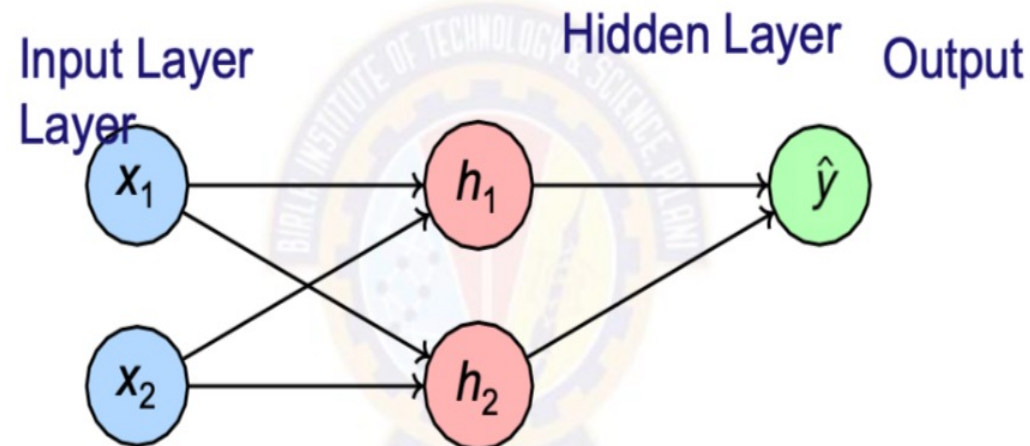
Decision boundary: $w_1x_1 + w_2x_2 + b = 0$ (a line/hyperplane)



Conclusion: We need models that can learn non-linear decision boundaries!

SOLUTION: MULTI-LAYER PERCEPTRON (MLP)

Key Idea: Stack multiple layers of neurons with non-linear activations



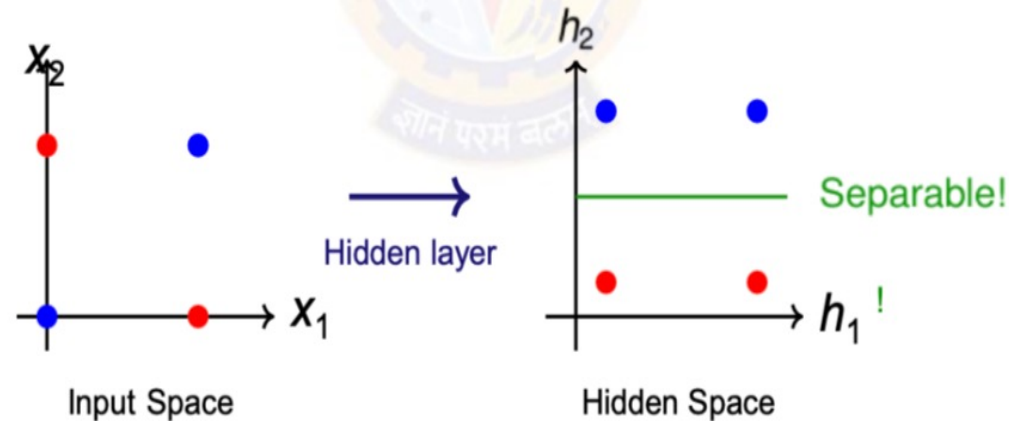
Non-linear activation functions
enable non-linear boundaries

MLP SOLVES XOR

Architecture: 2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output

How it works:

- Hidden layer creates new feature representations
- Each hidden unit learns a linear separator
- Output layer combines these separators
- Non-linear activation enables complex boundaries



WHAT IS NON-LINEARITY?

DEFINITION

A function f is **non-linear** if it does not satisfy the linearity property:

$$f(ax_1 + bx_2) \neq af(x_1) + bf(x_2) \quad (3)$$

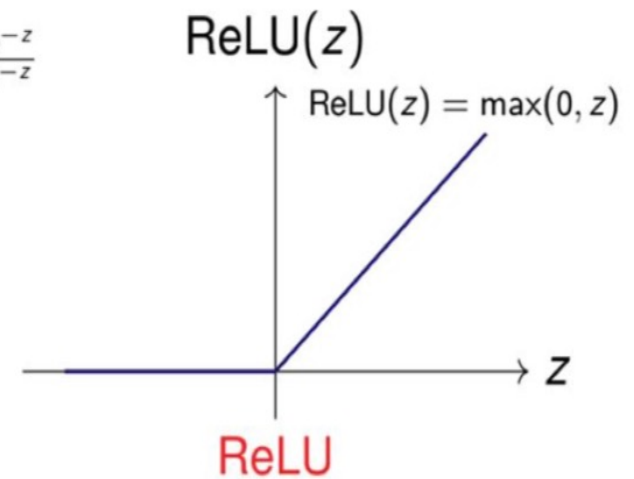
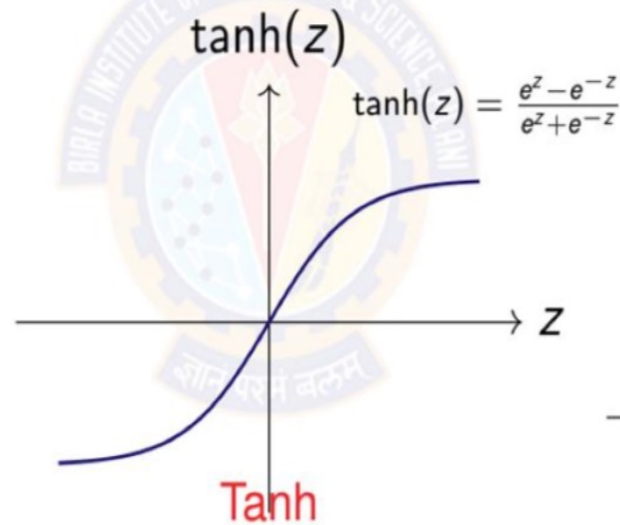
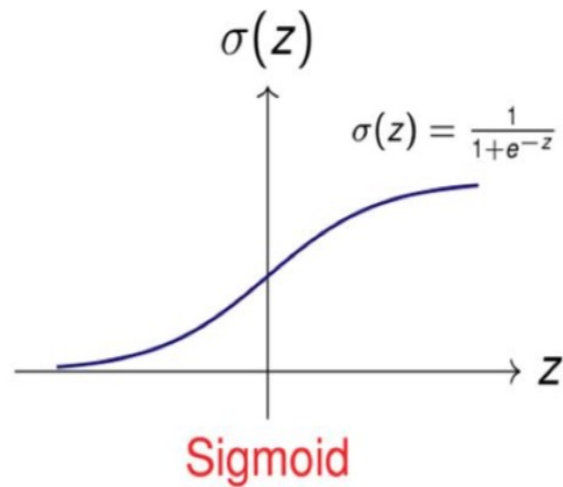
for all scalars a, b and inputs x_1, x_2 .

Why non-linearity matters:

- **Without non-linearity**, deep networks collapse to single-layer models
- **Non-linear activations** enable learning complex patterns

COMMON ACTIVATION FUNCTIONS

Activation functions introduce non-linearity



ACTIVATION FUNCTION PROPERTIES

Property	Sigmoid	Tanh	ReLU	Leaky ReLU
Range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$	$(-\infty, \infty)$
Zero-centered	x	✓	x	✓
Differentiable	✓	✓	x at 0	x at 0
Vanishing gradient	High	Moderate	No	No
Computational cost	High	High	Low	Low
Typical use	Output layer	Hidden layer	Hidden layer	Hidden layer

Modern preference: ReLU and variants for hidden layers

- Fast computation (simple thresholding)
- No vanishing gradient for positive values
- Sparse activation (biological plausibility)

WHY RELU WORKS WELL?

ReLU: $f(z) = \max(0, z)$

Advantages:

- ① Computational efficiency
 - › Simple comparison and multiplication
 - › Much faster than sigmoid/tanh (no exponentials)
- ② Avoids vanishing gradients
 - › Gradient is 1 for $z > 0$, flows unchanged through network
 - › Enables training of very deep networks
- ③ Sparse activation
 - › Many neurons output exactly 0
 - › More efficient representations
 - › Closer to biological neurons

Caveat: "Dying ReLU" problem when neurons always output 0

- Solution: Use Leaky ReLU, PReLU, or ELU variants

DEEP FEED-FORWARD NEURAL NETWORK (DFNN)

DEFINITION

A DFNN is a neural network with one or more **hidden layers** between input and output layers, where information flows forward through non-linear transformations.

Complete system requires four components:

- 1 **Data:** Input features and target outputs
- 2 **Model:** Multi-layer architecture with non-linear activations
- 3 **Objective Function:** Loss function to minimize
- 4 **Learning Algorithm:** Backpropagation with gradient descent

This is the same framework as linear models, but with added depth and non-linearity!

DATA REPRESENTATION

$$\text{Dataset: } \mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N \quad (4)$$

$$\text{Input Design Matrix: } \mathbf{X} \in \mathbb{R}^{N \times d} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \mathbf{x}^{(2)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_d^{(N)} \end{bmatrix} \quad (5)$$

$$\text{Target Matrix: } \mathbf{Y} \in \mathbb{R}^{N \times K} = \begin{bmatrix} \mathbf{y}^{(1)T} \\ \mathbf{y}^{(2)T} \\ \vdots \\ \mathbf{y}^{(N)T} \end{bmatrix} \quad (6)$$

where K is the output dimension (1 for regression, C for classification with C classes).

MINI-BATCH REPRESENTATION

For computational efficiency, we process data in mini-batches

$$\text{Mini-batch of size } B : \mathcal{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i \in \text{batch}} \quad (7)$$

$$\text{Mini-batch input matrix: } \mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B \times d} \quad (8)$$

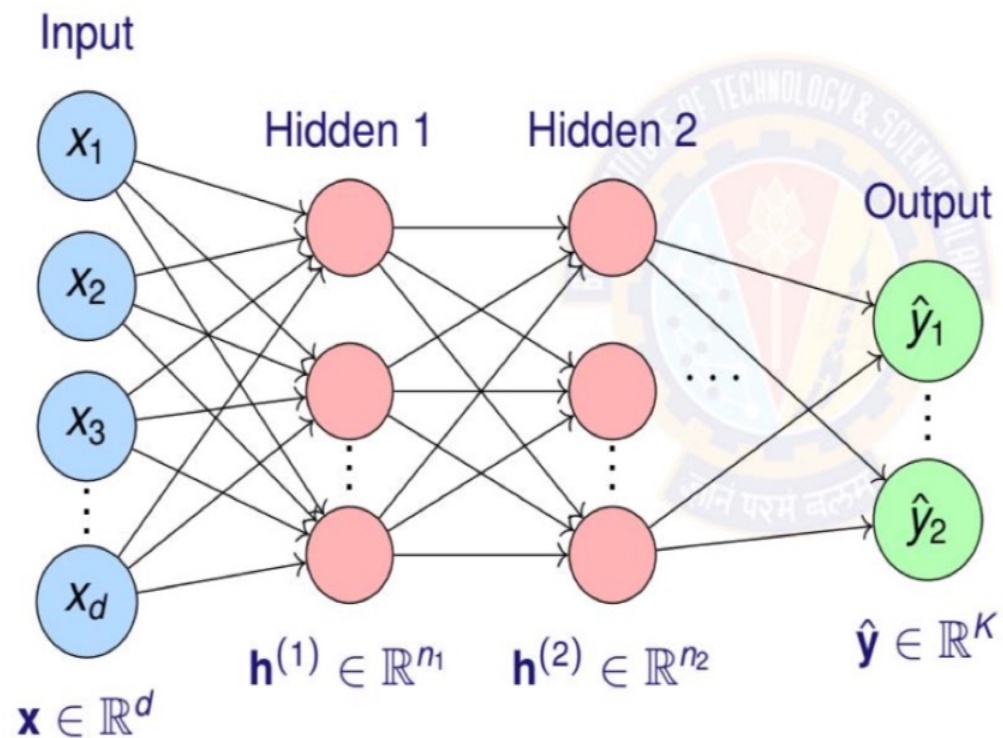
$$\text{Mini-batch target matrix: } \mathbf{Y}_{\mathcal{B}} \in \mathbb{R}^{B \times K} \quad (9)$$

Common batch sizes: $B \in \{32, 64, 128, 256\}$

- Balances computational efficiency with gradient stability
- Enables parallelization on GPUs

DFNN ARCHITECTURE

Network with L layers:



Notation:

- L : Number of layers (including output, excluding input)
- n_ℓ : Number of units in layer ℓ
- $d = n_0$: Input dimension
- $K = n_L$: Output dimension

LAYER-WISE COMPUTATION

For each layer $\ell = 1, 2, \dots, L$:

Parameters: $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_{\ell}}$ (weight matrix) (10)

$\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$ (bias vector) (11)

Initial condition: $\mathbf{h}^{(0)} = \mathbf{x}$ (input) (12)

Pre-activation (linear transformation): $\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \in \mathbb{R}^{n_{\ell}}$ (13)

Activation (non-linear transformation): $\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{n_{\ell}}$ (14)

Final output: $\hat{\mathbf{y}} = \mathbf{h}^{(L)}$ (15)

where $\sigma^{(\ell)}$ is the activation function for layer ℓ (applied element-wise).

TOTAL PARAMETERS IN NETWORK

Parameter count for layer ℓ :

$$\text{Parameters}^{(\ell)} = n_{\ell-1} \times n_{\ell} + n_{\ell} = n_{\ell}(n_{\ell-1} + 1) \quad (20)$$

Total parameters in network:

$$\text{Total} = \sum_{\ell=1}^L n_{\ell}(n_{\ell-1} + 1) \quad (21)$$

Example: Network with 3-layer architecture $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ neurons

$$\text{Layer 1: } 784 \times 256 + 256 = 200,960$$

$$\text{Layer 2: } 256 \times 128 + 128 = 32,896$$

$$\text{Layer 3: } 128 \times 10 + 10 = 1,290$$

$$\text{Total: } 235,146 \text{ parameters}$$

Deep networks can have millions to billions of parameters!

Deep Feedforward Neural Networks



- A neural network with more number of hidden layers is considered as a **deep neural network**. There are no feedback connections.
- Convolutional networks are examples of feedforward neural networks.

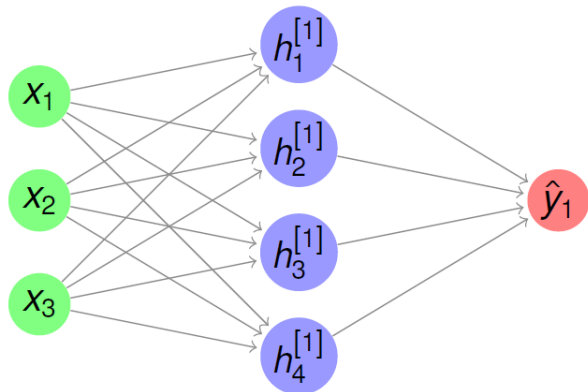


FIGURE: Shallow Neural Network

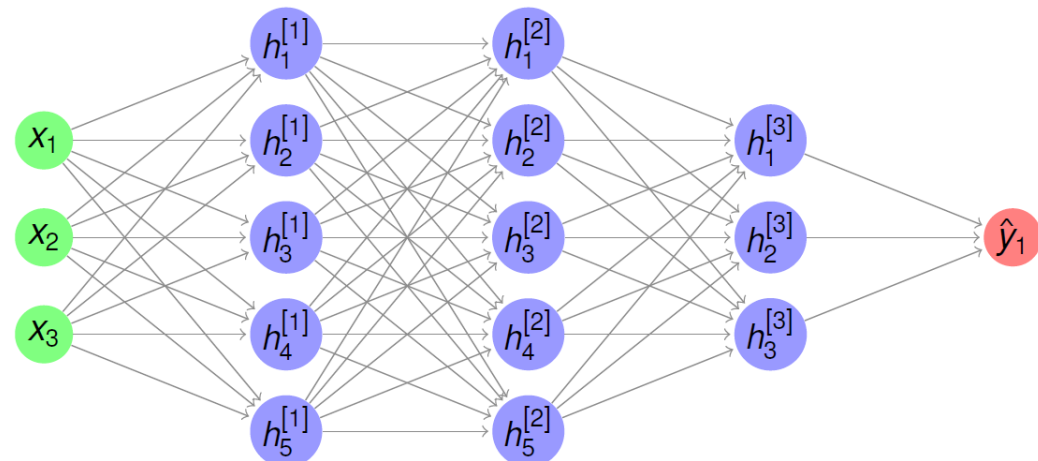


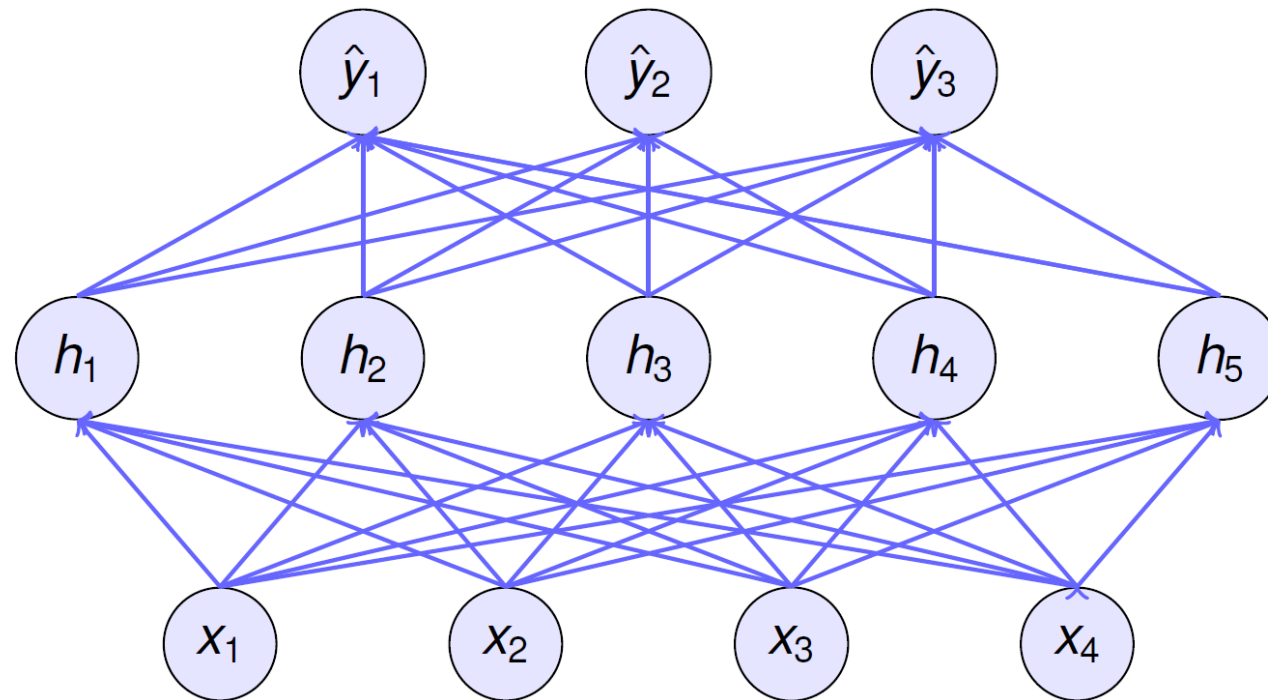
FIGURE: Deep Neural Network

Deep Feedforward Neural Networks



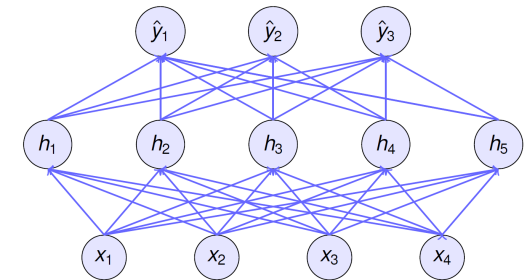
- With deep neural networks, use the data to jointly learn both a representation via hidden layers and a linear predictor that acts upon that representation.
- Add many hidden layers by stacking many fully-connected layers on top of each other. Each layer feeds into the layer above it, until we generate outputs.
- The first $(L - 1)$ layers learn the representation and the final layer is the linear predictor.

DNN Architecture



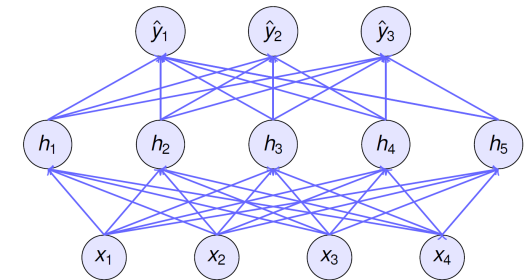
- DNN has 4 inputs, 3 outputs, and its hidden layer contains 5 hidden units.
- Number of layers in this DNN is 2.

DNN Architecture



- The layers are fully connected.
- Every input influences every neuron in the hidden layer.
- Each of the hidden neurons in turn influences every neuron in the output layer.
- The outputs of the hidden layer are called as **hidden representations** or hidden-layer variable or a hidden variable.

DNN Architecture



HIDDEN LAYERS – intermediate layers, desired output for each of these layers are not shown.

DEPTH – number of layers.

WIDTH – dimensionality of the hidden layers

DNN - General Strategy



- Design a DNN architecture of the network. Architecture depends on the problem / application / complexity of the decision boundary.
- Choose the activation functions that will be used to compute the hidden layer activations. The activation function is the same for all neurons in a layer. Activation function can change between layers.
- Choose the cost function. It depends on whether regression, binary classification or multi-class classification.
- Choose the optimizer algorithm, depends on the complexity and variance of the input data.
- Train the feedforward network. Learning in deep neural networks requires computing the gradients using the back-propagation algorithm.
- Evaluate the performance of the network.



The Flow of Computations in Neural Networks

- The flow of computations in a neural network goes in two ways:
 - **Left-to-right**: This is referred to as *forward propagation*, which results in computing the output of the network
 - **Right-to-left**: This is referred to as *back propagation*, which results in computing the gradients (or derivatives) of the parameters in the network
- The intuition behind this 2-way flow of computations can be explained through the concept of “computation graphs”

Computation Graphs



- Each node in the graph indicate a variable.
- An operation is a simple function of one or more variable.
- An operation is defined such that it returns only a single output variable.
- If a variable y is computed by applying an operation to a variable x , then we draw a directed edge from x to y .

Computation Graphs Example

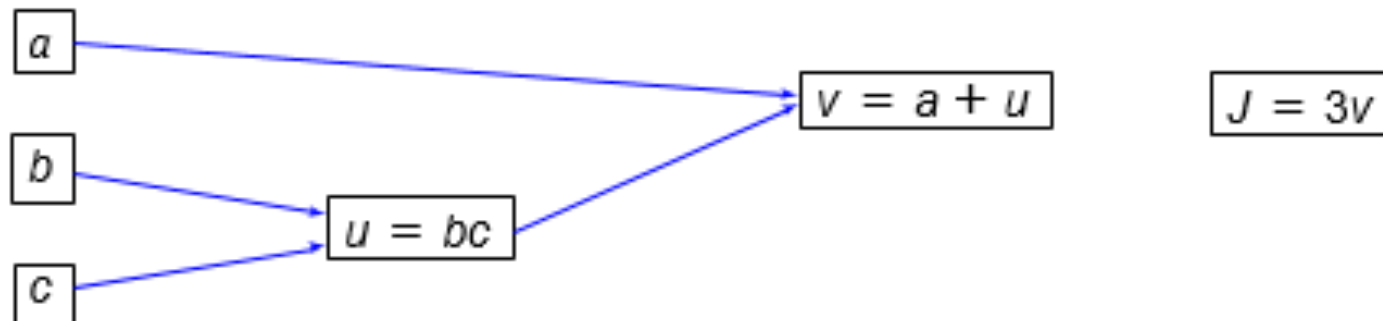
Forward Pass:

$$\text{Let } J(a, b, c) = 3(a + bc)$$

$$u = bc$$

$$v = a + u$$

$$\text{Then } J = 3v$$



Forward propagation allows computing J

Computation Graphs Example

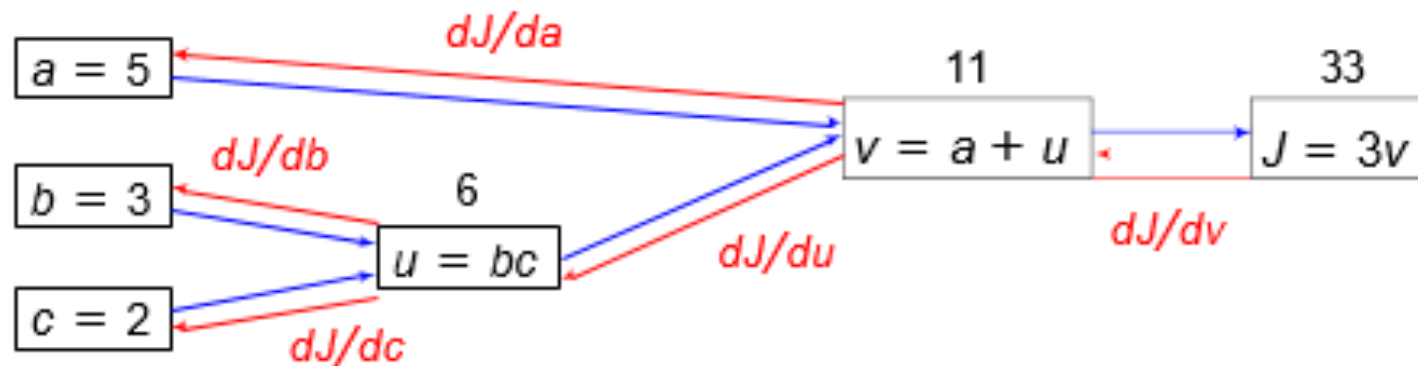
Backward Pass / Back propagation of gradients:

$$\text{Let } J(a, b, c) = 3(a + bc)$$

$$u = bc$$

$$v = a + u$$

$$\text{Then } J = 3v$$



To compute the derivative of J with respect to v , we went *back* to v , nudged it, and measured the corresponding resultant increase on J



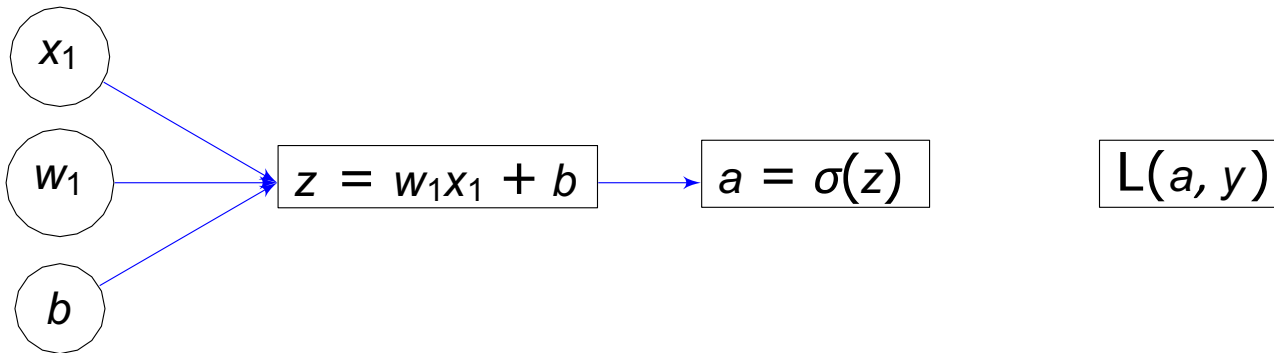
Computation Graph – Binary Classification

Forward Pass:

$$z = w^T X + b$$

$$a = y = \sigma(z)$$

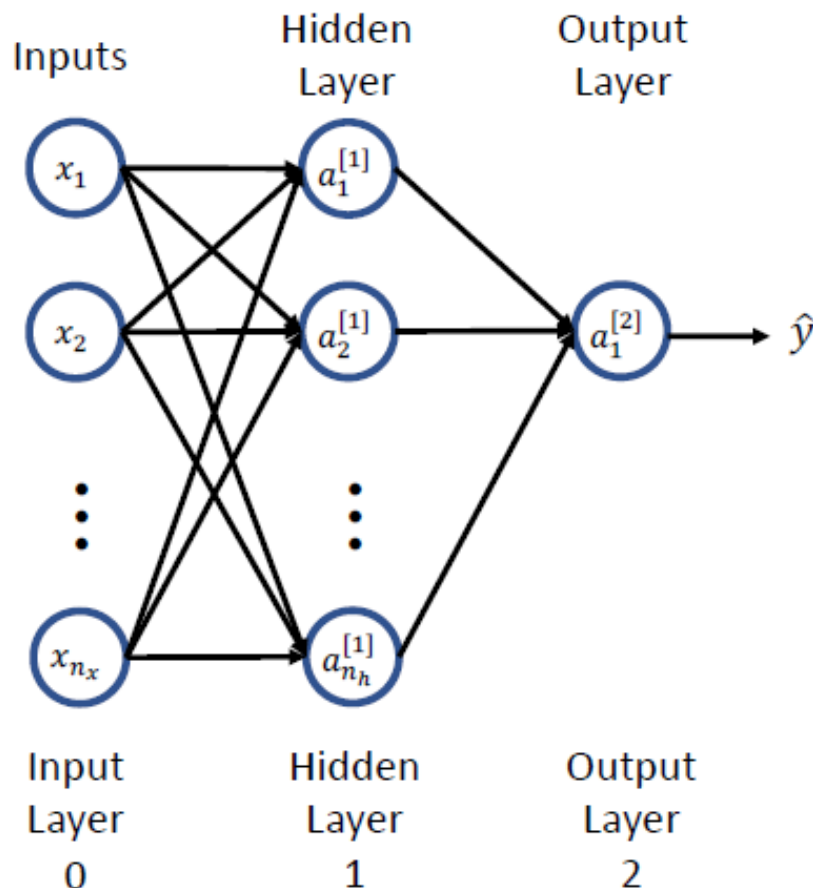
$$L(a, y) = -[y \log a + (1 - y) \log(1 - a)]$$



Formalizing Neural Network Construction

$$Y = f_N(\mathbf{W}_N f_{N-1}(\dots f_2(\mathbf{W}_2 f_1(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) \dots) + \mathbf{b}_N)$$

Let's consider a 2-layer Network



Output of one activation unit

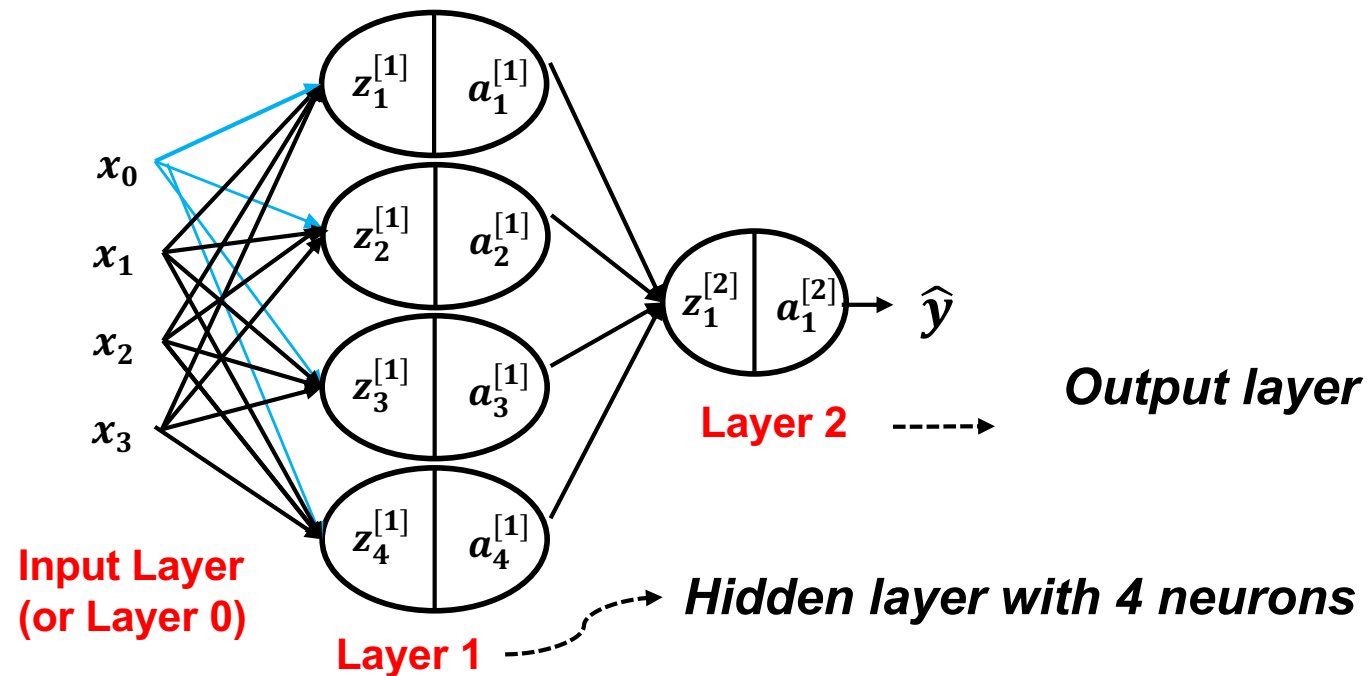
$$z_i^{[l]} = \sum_{j=1}^{n_h^{[l-1]}} w_{i,j}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

$$a_i^{[l]} = g(z_i^{[l]})$$

Where

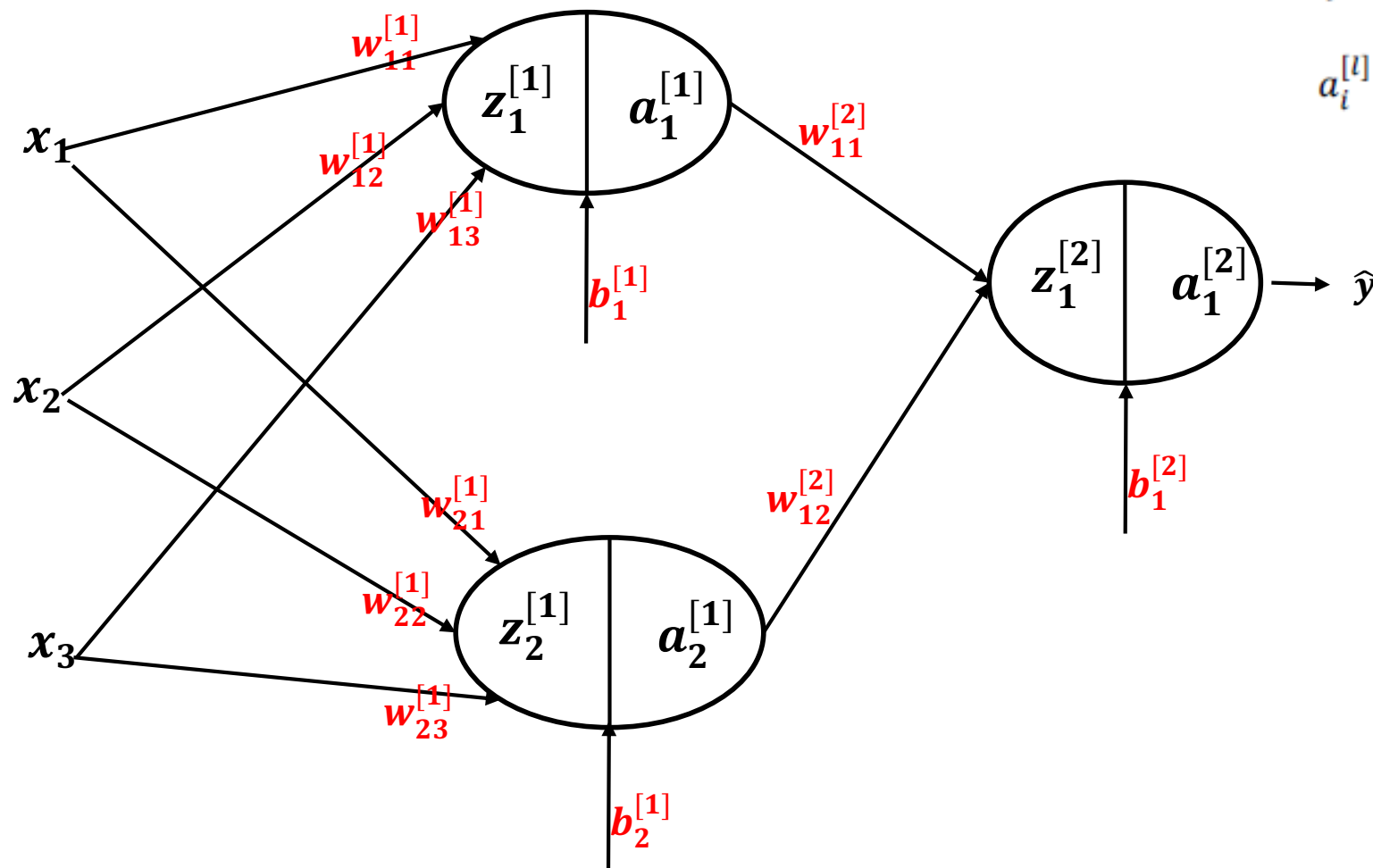
- $g()$ = activation function
- $w^{[l]}$ = weight parameter of input $a^{[l-1]}$ going to unit $a^{[l]}$
- n_x = number of features (model inputs)
- n_h = number neurons/units in hidden layer
- $x_i = a_i^{[0]}$

2-layer NN



Interestingly,
neural
networks *learn*
their own
features!

Example (without vectorization)

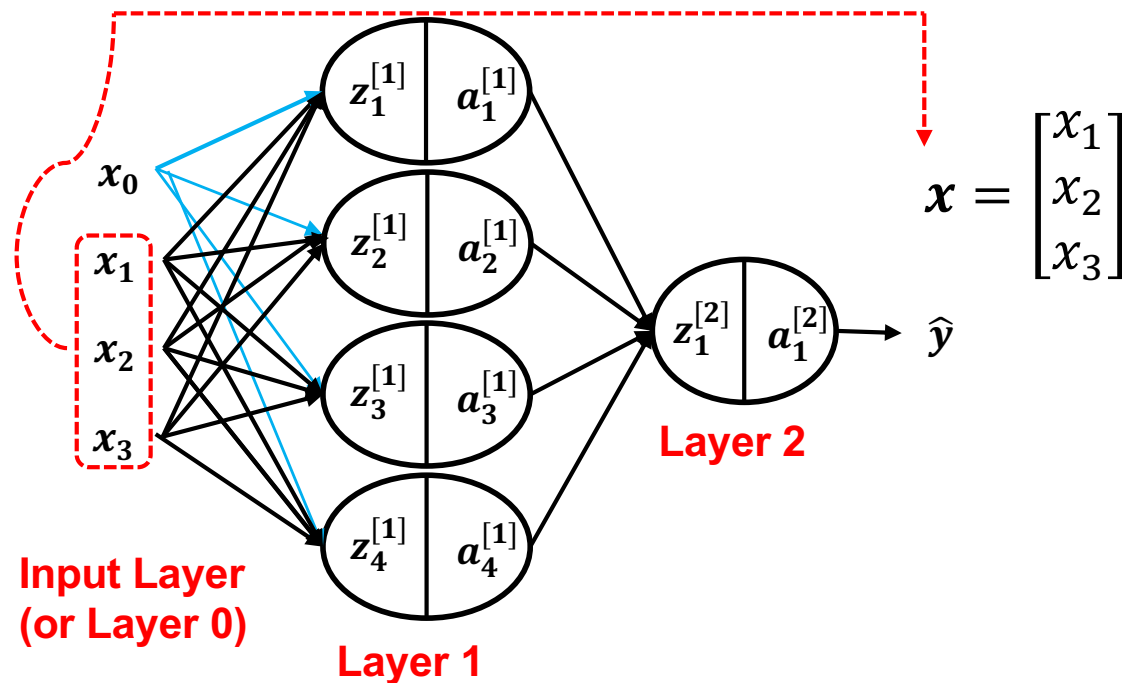


$$z_i^{[l]} = \sum_{j=1}^{n_h^{[l-1]}} w_{i,j}^{[l]} a_j^{[l-1]} + b_i^{[l]}$$

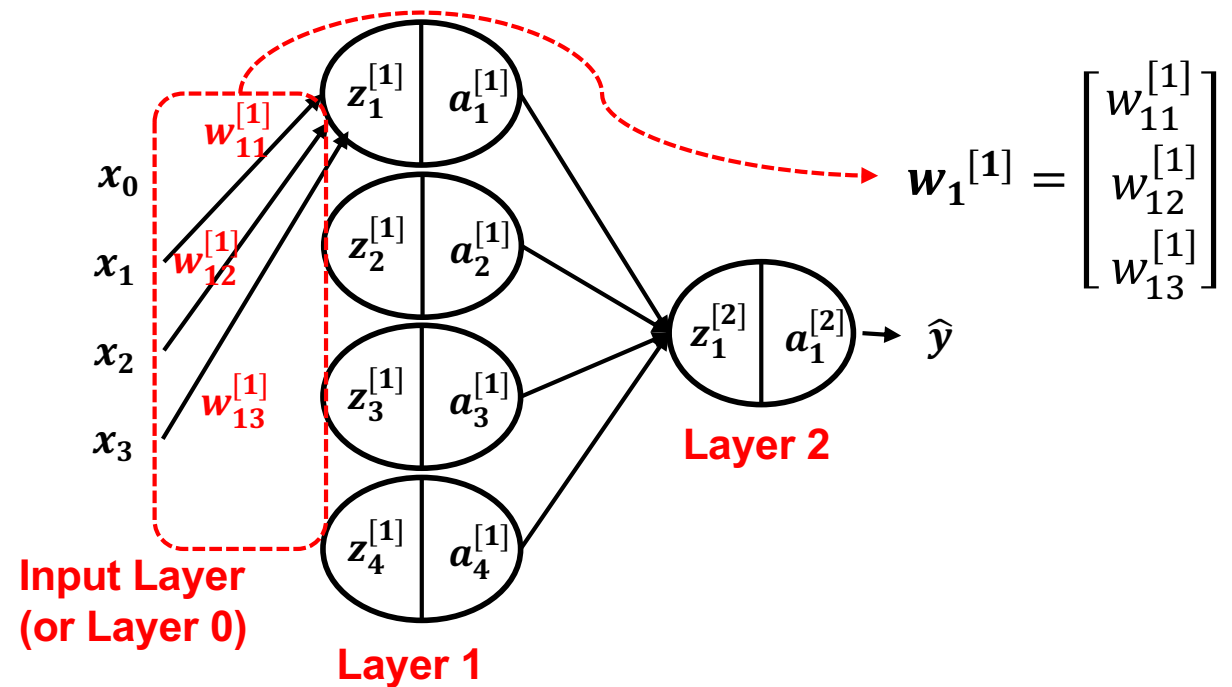
$$a_i^{[l]} = g(z_i^{[l]})$$

Vectorizing Input and All Variables

To help develop an efficient learning algorithm, let us *vectorize* (represent in vectors & matrices) the input and the variables involved



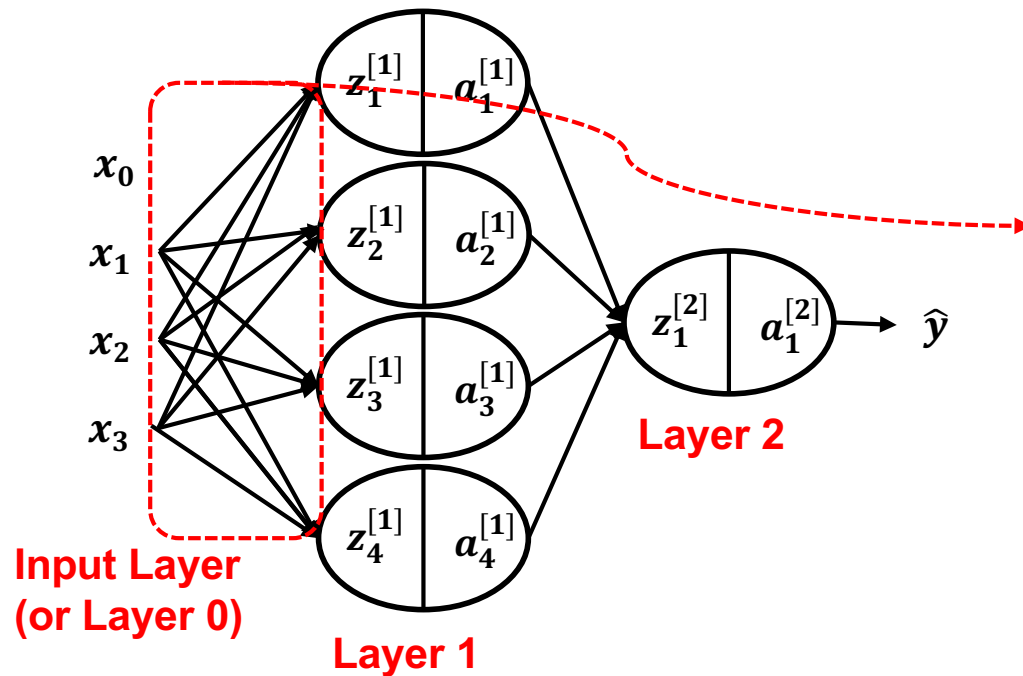
Vectorizing Input and All Variables



Vectorizing Input and All Variables

$$\mathbf{w}_1^{[1]} = \begin{bmatrix} w_{11}^{[1]} \\ w_{12}^{[1]} \\ w_{13}^{[1]} \end{bmatrix}$$

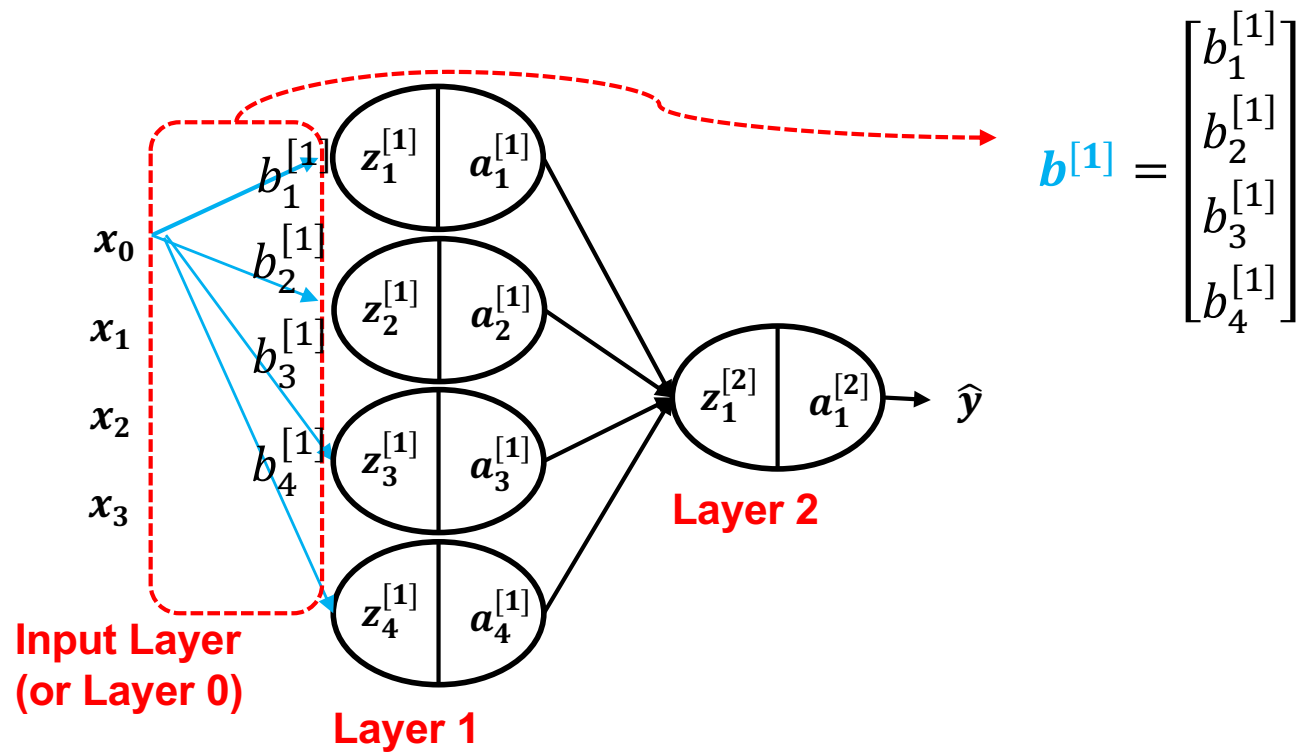
$$\mathbf{w}^{[1]} = \underbrace{\begin{bmatrix} \dots & \mathbf{w}_1^{[1]T} & \dots \\ \dots & \mathbf{w}_2^{[1]T} & \dots \\ \dots & \mathbf{w}_3^{[1]T} & \dots \\ \dots & \mathbf{w}_4^{[1]T} & \dots \end{bmatrix}}_{4 \times 3}$$



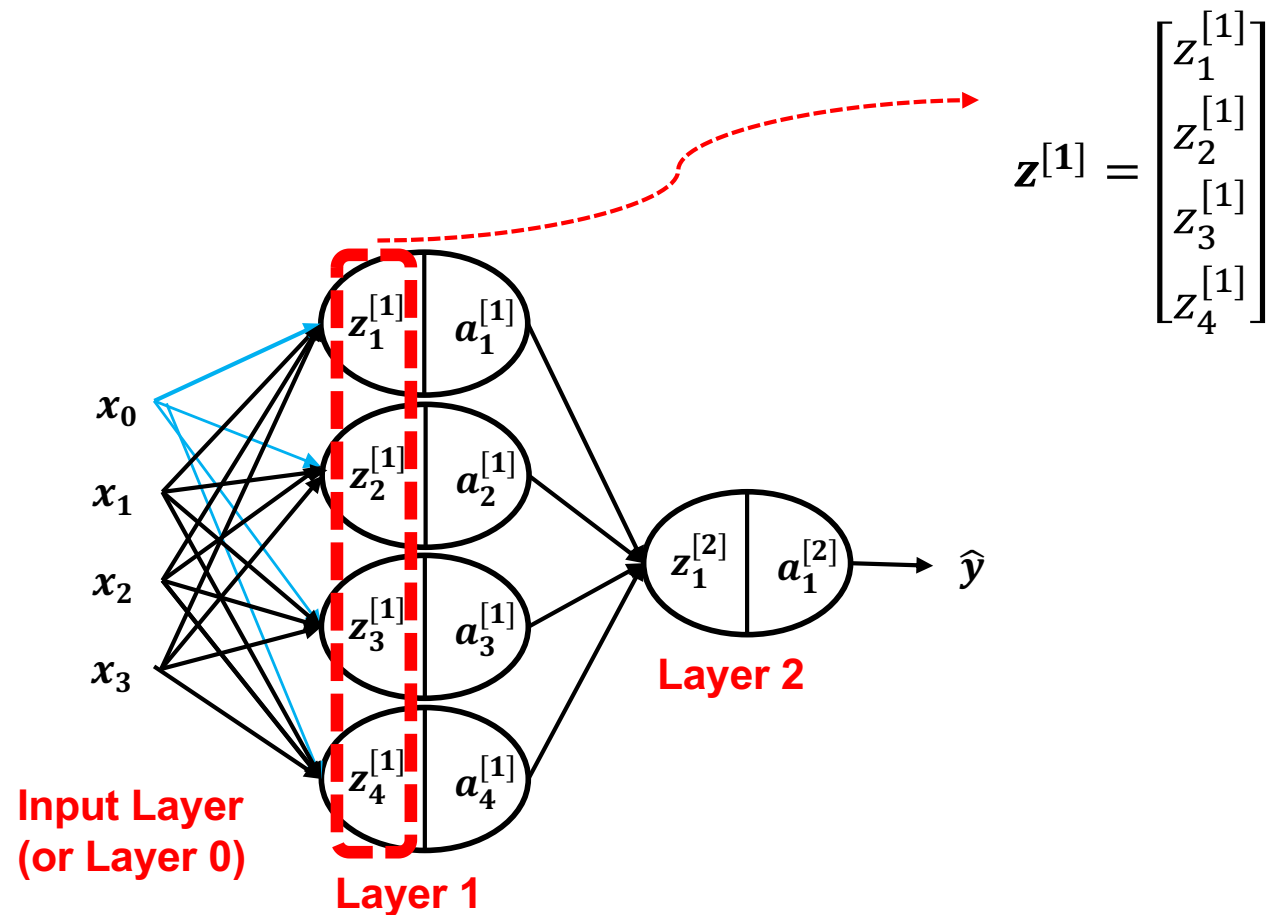
$$\mathbf{w}^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \\ w_{31}^{[1]} & w_{32}^{[1]} & w_{33}^{[1]} \\ w_{41}^{[1]} & w_{42}^{[1]} & w_{43}^{[1]} \end{bmatrix}$$

Dimension of $\mathbf{w}^{[1]} = (4, 3)$

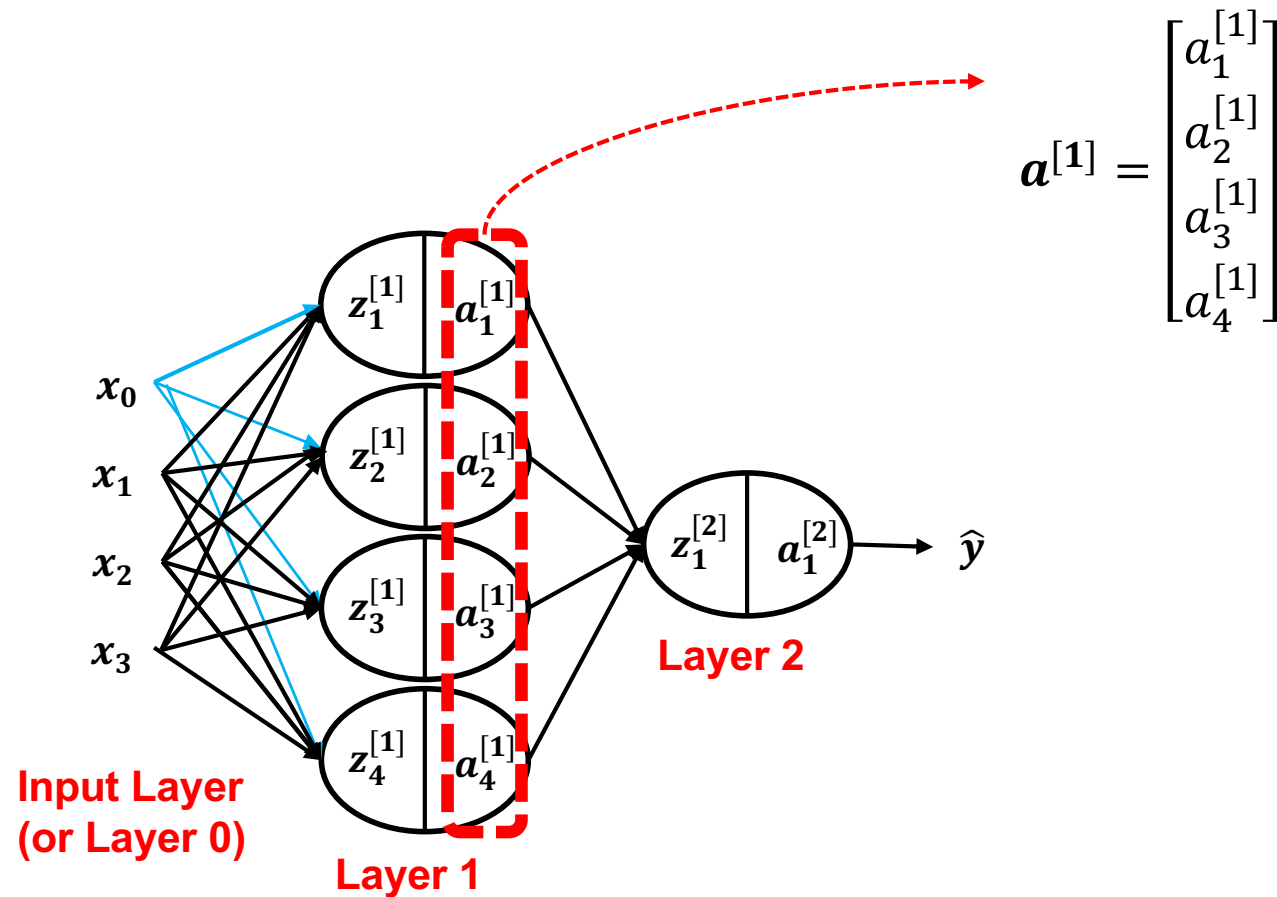
Vectorizing Input and All Variables



Vectorizing Input and All Variables

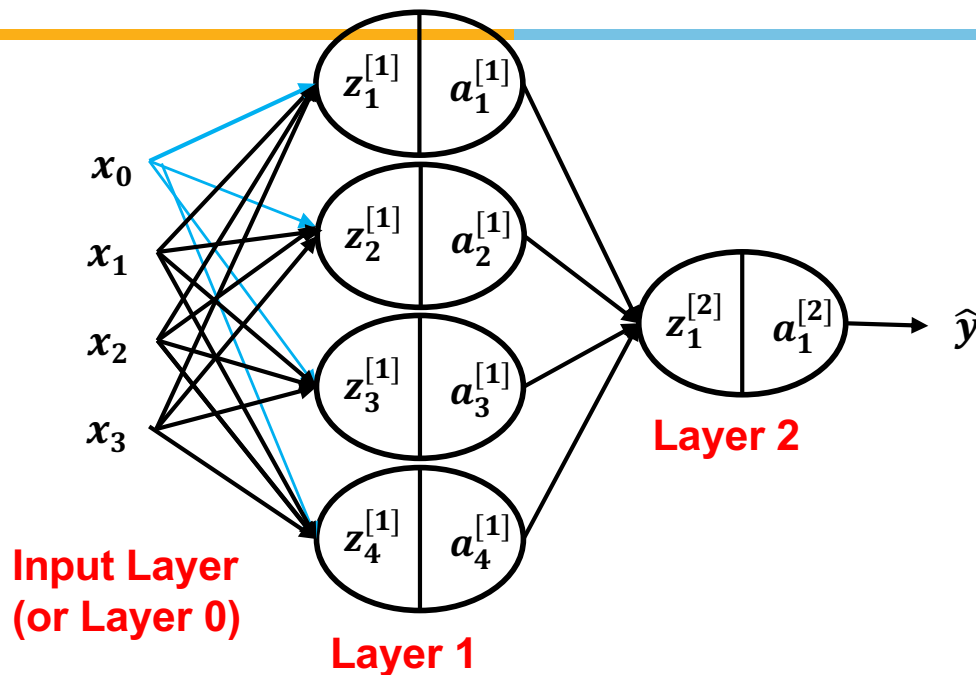


Vectorizing Input and All Variables



Vectorizing Input and All Variables

$$z^{[1]} = w^{[1]}x + b^{[1]}$$



$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & w_{13}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & w_{23}^{[1]} \\ w_{31}^{[1]} & w_{32}^{[1]} & w_{33}^{[1]} \\ w_{41}^{[1]} & w_{42}^{[1]} & w_{43}^{[1]} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

In matrix form

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \underbrace{\begin{bmatrix} \dots & w_1^{[1]T} & \dots \\ \dots & w_2^{[1]T} & \dots \\ \dots & w_3^{[1]T} & \dots \\ \dots & w_4^{[1]T} & \dots \end{bmatrix}}_{4 \times 3} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{3 \times 1} + \underbrace{\begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}}_{4 \times 1}$$

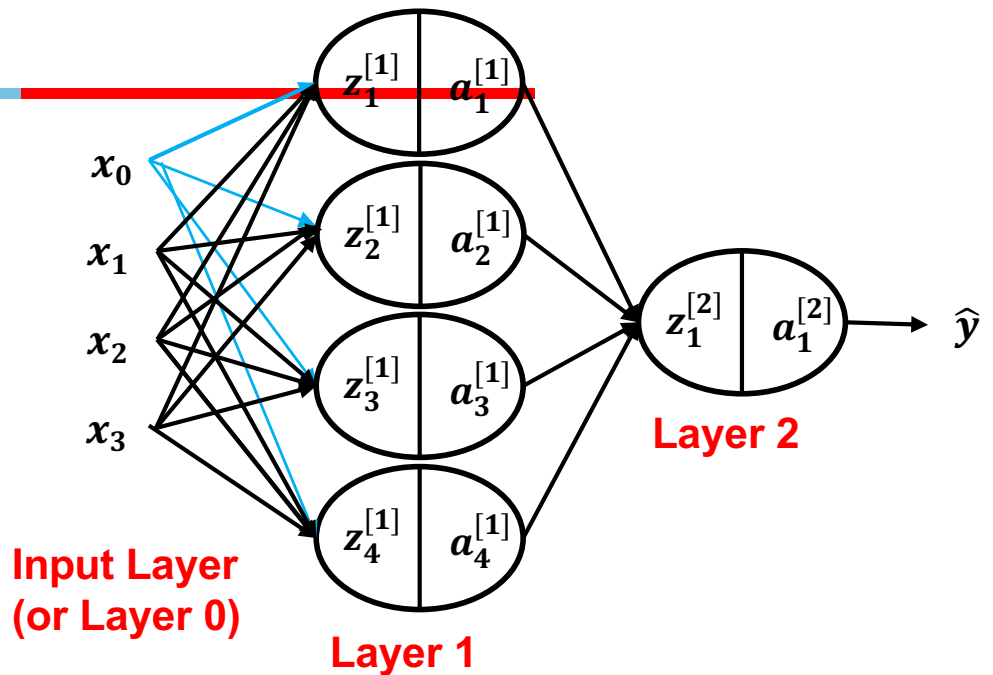
$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 + b_1^{[1]} \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 + b_2^{[1]} \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 + b_3^{[1]} \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 + b_4^{[1]} \end{bmatrix}$$

Vectorizing Input and All Variables



$$a^{[1]} = \sigma(z^{[1]})$$

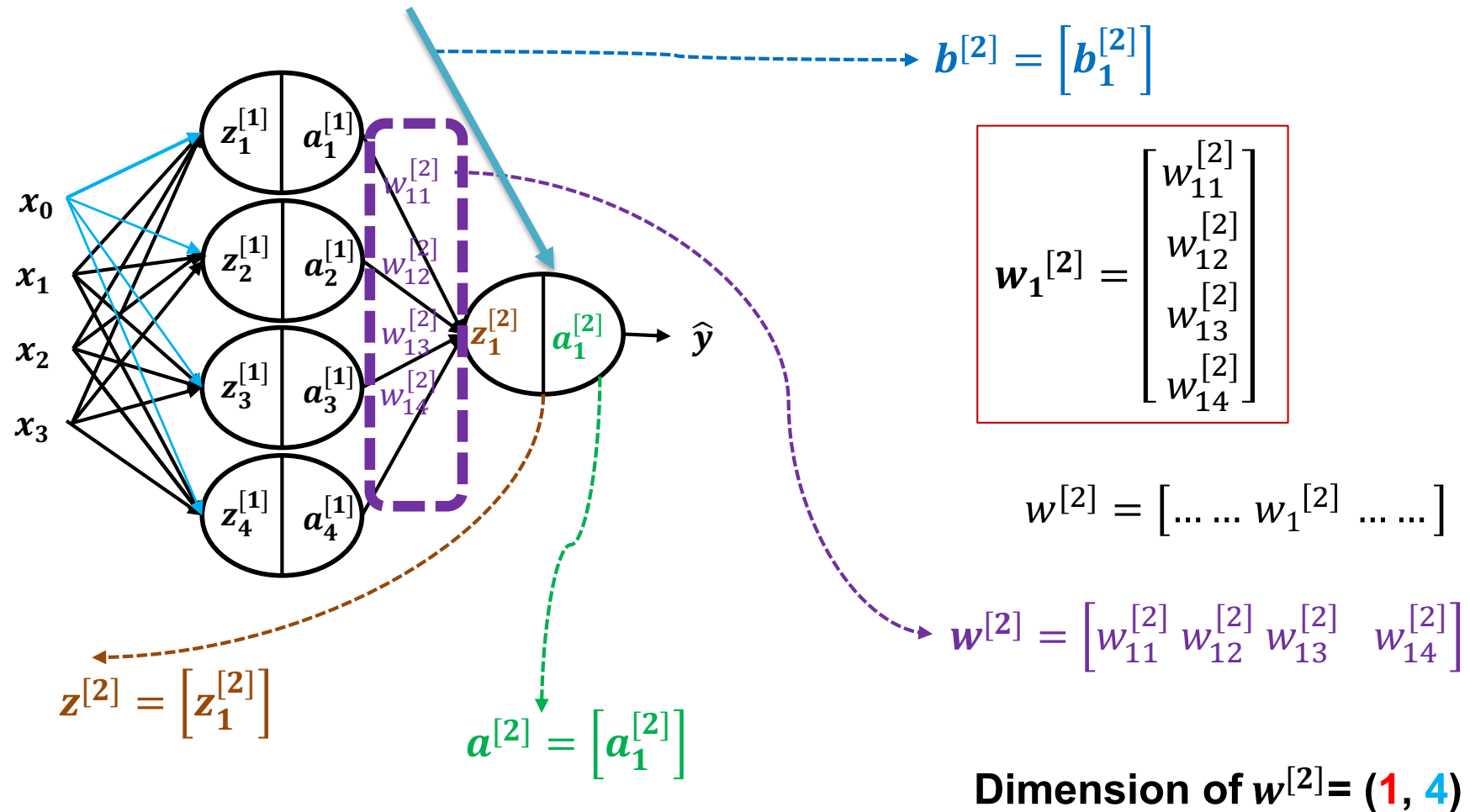
$$z^{[1]} = w^{[1]}x + b^{[1]}$$



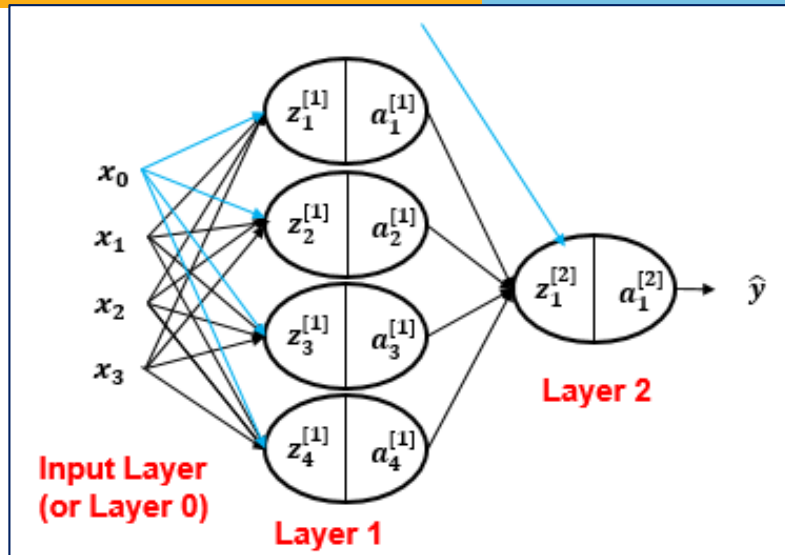
$$z^{[1]} = \begin{bmatrix} w_{11}^{[1]}x_1 + w_{12}^{[1]}x_2 + w_{13}^{[1]}x_3 + b_1^{[1]} \\ w_{21}^{[1]}x_1 + w_{22}^{[1]}x_2 + w_{23}^{[1]}x_3 + b_2^{[1]} \\ w_{31}^{[1]}x_1 + w_{32}^{[1]}x_2 + w_{33}^{[1]}x_3 + b_3^{[1]} \\ w_{41}^{[1]}x_1 + w_{42}^{[1]}x_2 + w_{43}^{[1]}x_3 + b_4^{[1]} \end{bmatrix}$$

A green box highlights the first row of the vector $z^{[1]}$, which is connected by a dashed green arrow to the text $(z_1^{[1]}) \rightarrow a_1^{[1]} = \sigma(z_1^{[1]})$.

Vectorizing Input and All Variables



Vectorizing Input and All Variables



$$a^{[2]} = \sigma(z^{[2]})$$

$$z^{[2]} = w^{[2]}x + b^{[2]}$$

$$= w^{[2]}a^{[1]} + b^{[2]}$$

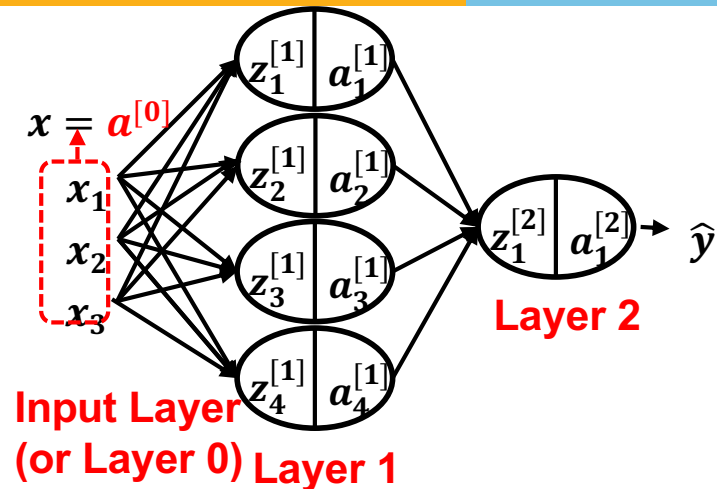
$$= \begin{bmatrix} w_{11}^{[2]} & w_{12}^{[2]} & w_{13}^{[2]} & w_{14}^{[2]} \end{bmatrix} \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ a_3^{[1]} \\ a_4^{[1]} \end{bmatrix} + \begin{bmatrix} b_1^{[2]} \end{bmatrix}$$

$$= \left[w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} \right] + \begin{bmatrix} b_1^{[2]} \end{bmatrix}$$

$$a_1^{[2]} = \sigma(z_1^{[2]})$$

$$z_1^{[2]}$$

Vectorizing Input and All Variables



Forward propagation equations

$$z^{[1]} = w^{[1]}x + b^{[1]}$$

$$a^{[1]} = \sigma(z^{[1]})$$

$$z^{[2]} = w^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = \sigma(z^{[2]})$$

For l^{th} layer

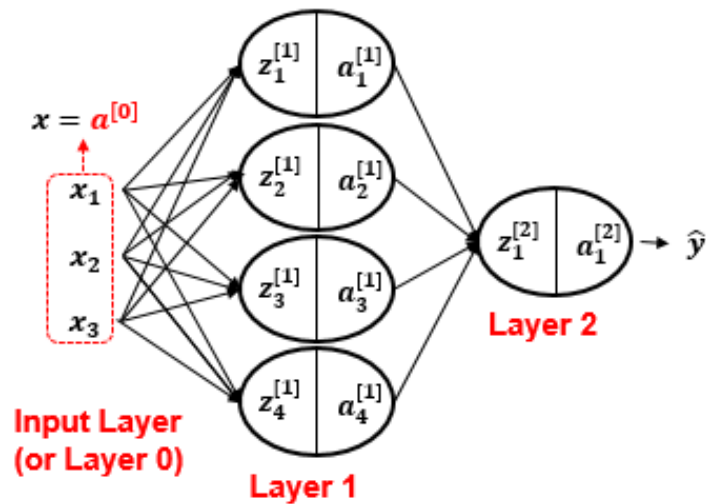
$$z^{[l]} = w^{[l]}x + b^{[l]}$$

$$a^{[l]} = \sigma(z^{[l]})$$

But, this assumes only 1 training example!

Vectorizing Input and All Variables

Assuming m examples



for $i = 1$ to m :

$$z^{[1]}(i) = w^{[1]} a^{[0]}(i) + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = w^{[2]} a^{[1]}(i) + b^{[2]}$$

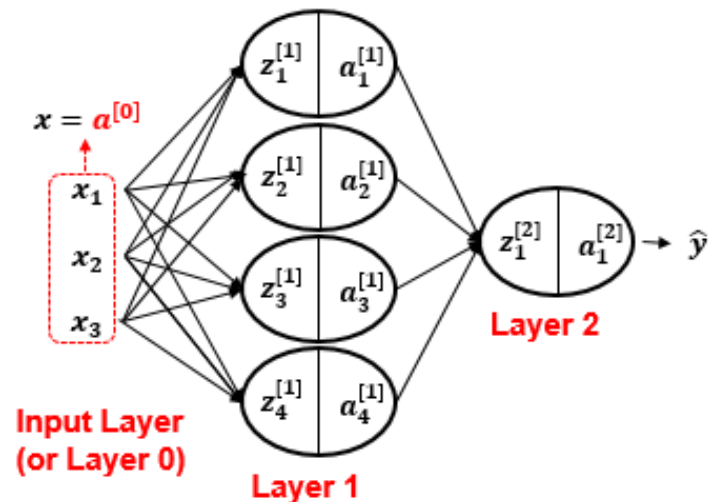
$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

Refers to the i^{th} example in the training dataset

Vectorizing Input and All Variables



Assuming m examples

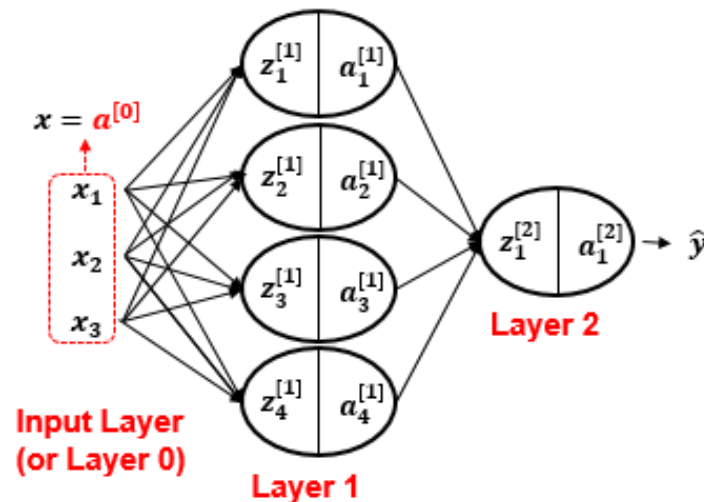


But, loops in general slow down programs; hence, it is better to further ***vectorize*** the implementation in order to avoid any loop, whenever possible

Vectorizing Input and All Variables

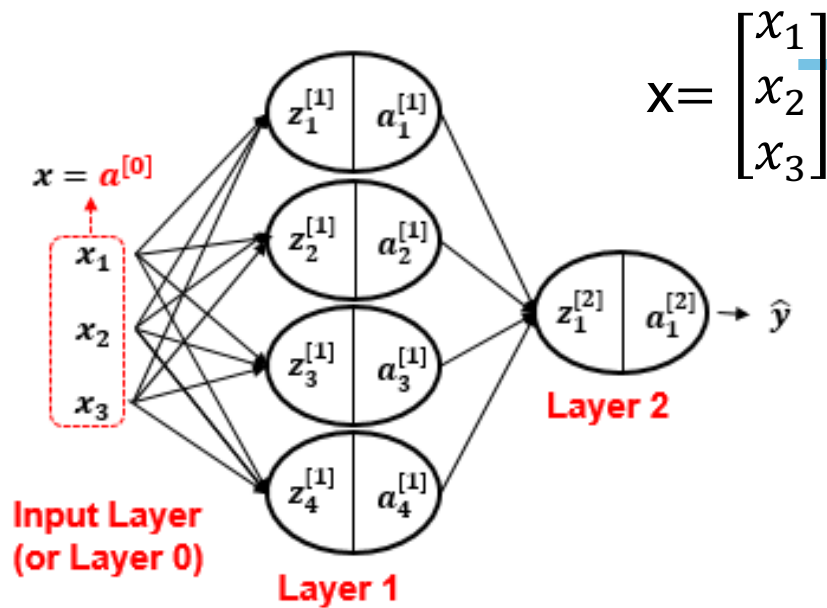


Assuming m examples



To this end, we can simply stack all x vectors (or $a^{[0]}$ vectors), z vectors, and a vectors in different matrices of every layer!

Vectorizing Input and All Variables



$$x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Assuming m examples

$$X = \begin{bmatrix} X^{(1)} & X^{(2)} & \dots & X^{(m)} \end{bmatrix}$$

$$X = \begin{bmatrix} x_1^{(1)} & x_1^{(2)} & \dots & x_1^{(m)} \\ x_2^{(1)} & x_2^{(2)} & \dots & x_2^{(m)} \\ x_3^{(1)} & x_3^{(2)} & \dots & x_3^{(m)} \end{bmatrix}$$

This vector represents the *first* example in the training dataset

If we denote x as $a^{[0]}$

No. of features = $n_x = 3$

$$X = A^{[0]} = \begin{bmatrix} a_1^{[0](1)} & a_1^{[0](2)} & \dots & a_1^{[0](m)} \\ a_2^{[0](1)} & a_2^{[0](2)} & \dots & a_2^{[0](m)} \\ a_3^{[0](1)} & a_3^{[0](2)} & \dots & a_3^{[0](m)} \end{bmatrix}$$

$$A^{[0]} \text{ shape} = (n_x, m)$$

Vectorizing Input and All Variables



Assuming m examples

Assuming m examples

$$\mathbf{Z}^{[1]} = \begin{bmatrix} \mathbf{z}^{[1]}_{(1)} & \mathbf{z}^{[1]}_{(2)} & \dots & \mathbf{z}^{[1]}_{(m)} \\ | & | & & | \end{bmatrix}$$

No. of hidden units at layer 1 = $n_h = 4$

$\mathbf{Z}^{[1]}$ shape = (n_h, m)

$$\mathbf{Z}^{[1]} = \mathbf{w}^{[1]} \mathbf{A}^{[0]} + \mathbf{b}^{[1]}$$

$$\mathbf{Z}^{[1]} = \text{matmul}(\mathbf{w}^{[1]}, \mathbf{A}^{[0]}) + \mathbf{b}^{[1]}$$

$$\mathbf{Z}^{[1]} = \begin{bmatrix} z_1^{[1]}(1) & z_1^{[1]}(2) & & z_1^{[1]}(m) \\ z_2^{[1]}(1) & z_2^{[1]}(2) & & z_2^{[1]}(m) \\ z_3^{[1]}(1) & z_3^{[1]}(2) & \dots & z_3^{[1]}(m) \\ z_4^{[1]}(1) & z_4^{[1]}(2) & & z_4^{[1]}(m) \end{bmatrix}$$

Vectorizing Input and All Variables



Assuming m examples

Assuming m examples

$$A^{[1]} = \begin{bmatrix} A^{1} & A^{[1](2)} & \dots & A^{[1](m)} \\ | & | & & | \end{bmatrix}$$

$A^{[1]}$ shape = (n_h, m)

$$A^{[1]} = \sigma(Z^{[1]})$$

$$A^{[1]} = \begin{bmatrix} a_1^{1} & a_1^{[1](2)} & & a_1^{[1](m)} \\ a_2^{1} & a_2^{[1](2)} & & a_2^{[1](m)} \\ a_3^{1} & a_3^{[1](2)} & \dots & a_3^{[1](m)} \\ a_4^{1} & a_4^{[1](2)} & & a_4^{[1](m)} \end{bmatrix}$$

Vectorizing Input and All Variables



Assuming m examples

Assuming m examples

$Z^{[2]}$ shape = (1,m)

$$Z^{[2]} = \begin{bmatrix} z_1^{[2](1)} & z_1^{2} & \dots & z_1^{[2](m)} \end{bmatrix}$$

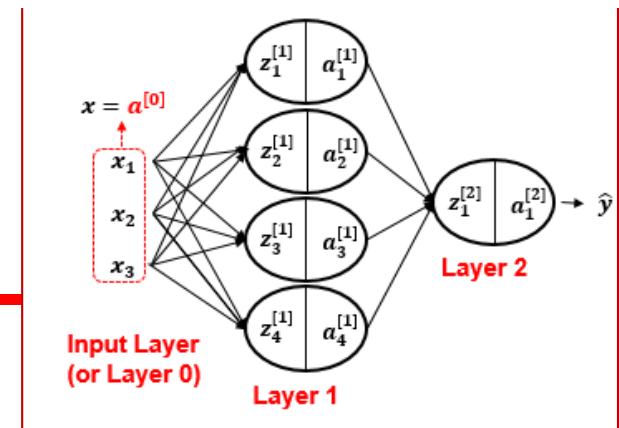
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$A^{[2]}$ shape = (1,m)

$$A^{[2]} = \begin{bmatrix} a_1^{[2](1)} & a_1^{2} & \dots & a_1^{[2](m)} \end{bmatrix}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

Forward propagation equations



for $i = 1$ *to* n :

$$z^{[1]}(i) = w^{[1]} a^{[0]}(i) + b^{[1]}$$

$$a^{[1]}(i) = \sigma(z^{[1]}(i))$$

$$z^{[2]}(i) = w^{[2]} a^{[1]}(i) + b^{[2]}$$

$$a^{[2]}(i) = \sigma(z^{[2]}(i))$$

Before Vectorization

No Explicit Loop!

$$Z^{[1]} = w^{[1]} A^{[0]} + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = w^{[2]} A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma(Z^{[2]})$$

After Vectorization

Matrix dimensions

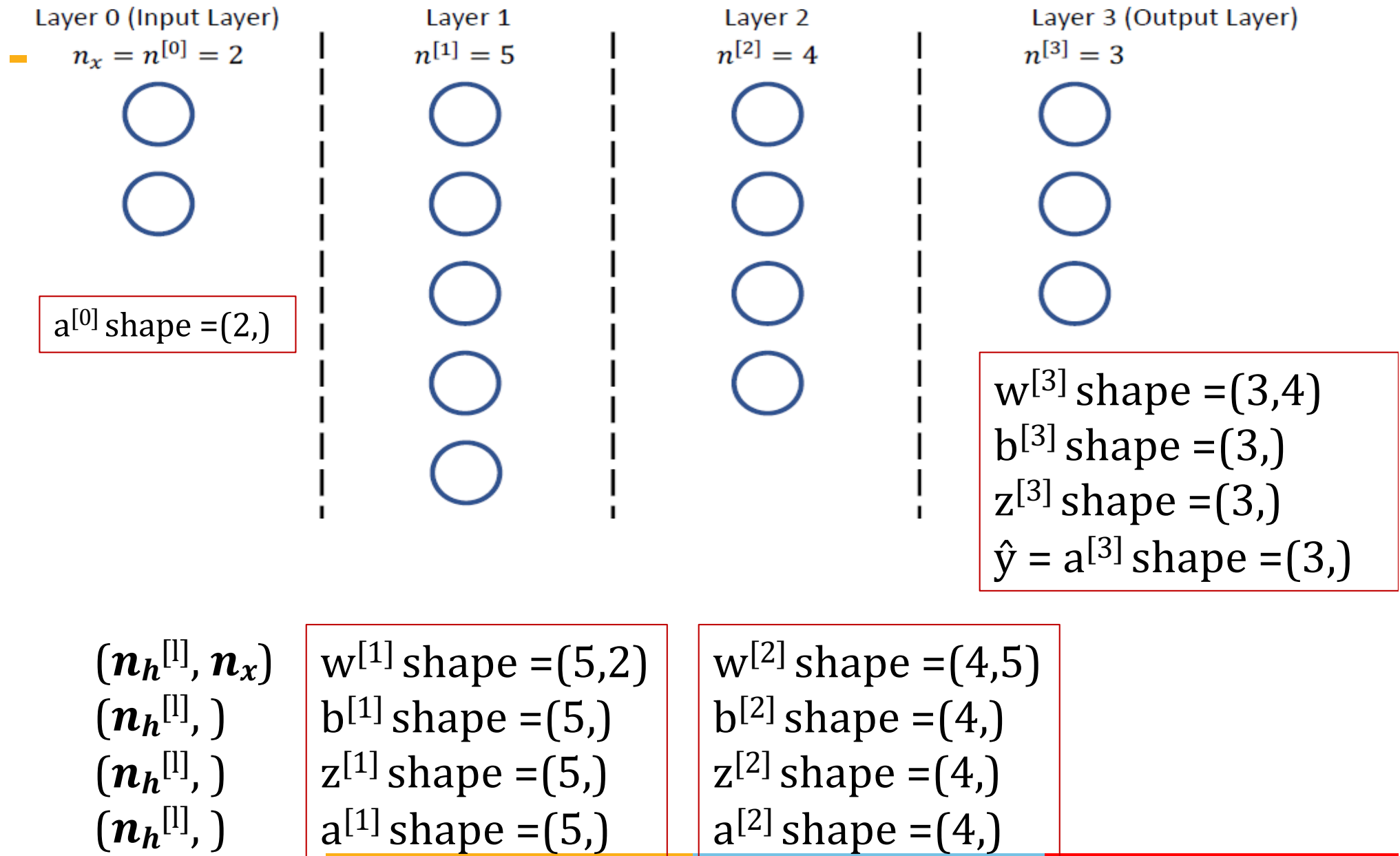
Notations

- L : Number of layers
- $n^{[l]}$: Number of units in layer l
- $w^{[l]}$: Weights for layer l
- $b^{[l]}$: Bias for layer l
- $z^{[l]}$: Hypothesis for layer l
- $g^{[l]}$: Activation Function used for layer l
- $a^{[l]}$: Activation for layer l

Matrix Dimensions

- $W^{[l]} : n^{[l]} \times n^{[l-1]}$
 - $b^{[l]} : n^{[l]} \times 1$
 - $z^{[l]} : n^{[l]} \times 1$
 - $a^{[l]} : n^{[l]} \times 1$
 - $dW^{[l]} : n^{[l]} \times n^{[l-1]}$
 - $db^{[l]} : n^{[l]} \times 1$
 - $dz^{[l]} : n^{[l]} \times 1$
 - $da^{[l]} : n^{[l]} \times 1$
-

Dimensions for 1 training example:



Forward Step



- Given some [initial] values for the parameters, we can compute **the forward pass**, layer by layer.
- Forward pass is basically **L matrix multiplications**, each followed by an activation function.
- Matrix multiplication can be done efficiently with GPUs.
 - Therefore, **forward pass is somewhat fast**.
- Complexity of forward pass, **linear of depth $O(L)$** .



Practice problems



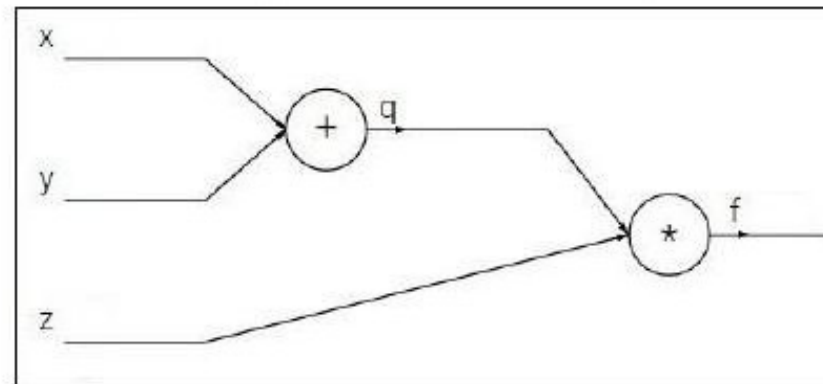


Example 1 Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

Computational Graph for Back Propagation

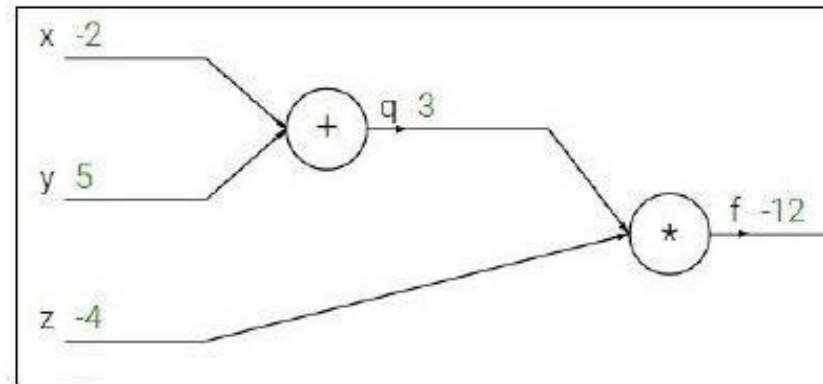
$$f(x, y, z) = (x + y)z$$



Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

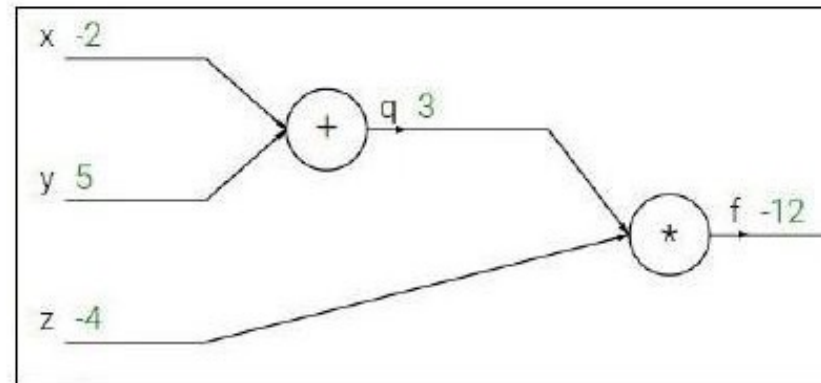
e.g. $x = -2, y = 5, z = -4$



Computational Graph for Back Propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

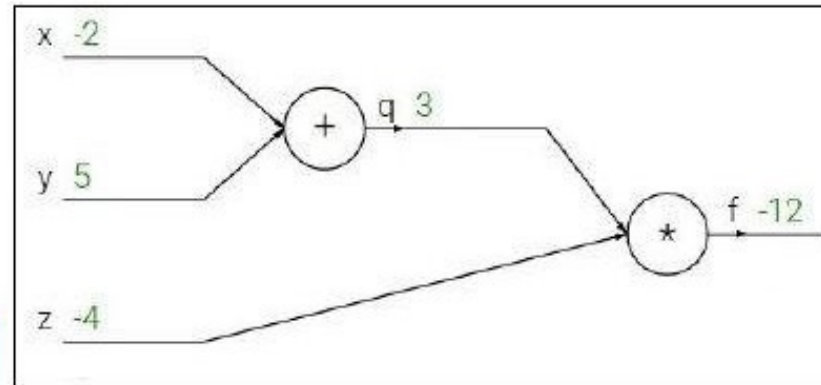
Computational Graph for Back Propagation



$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Computational Graph for Back Propagation

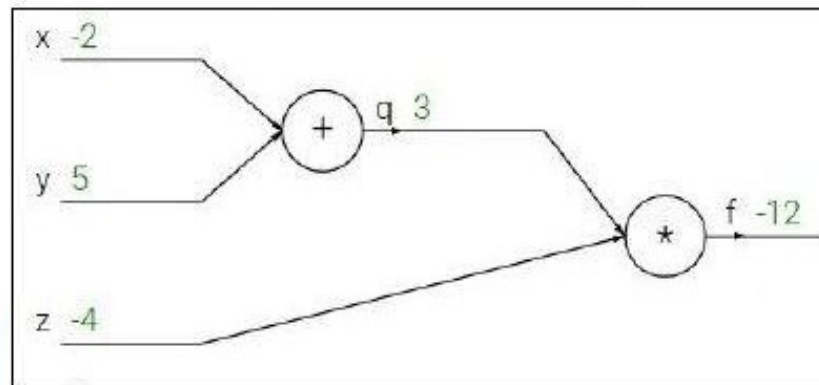
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

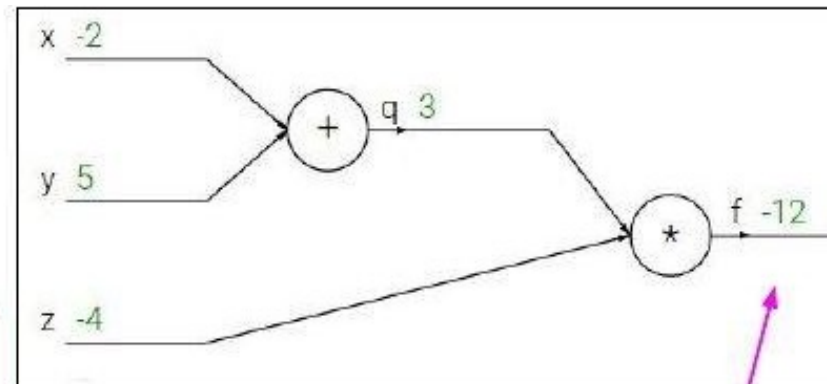
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

Computational Graph for Back Propagation



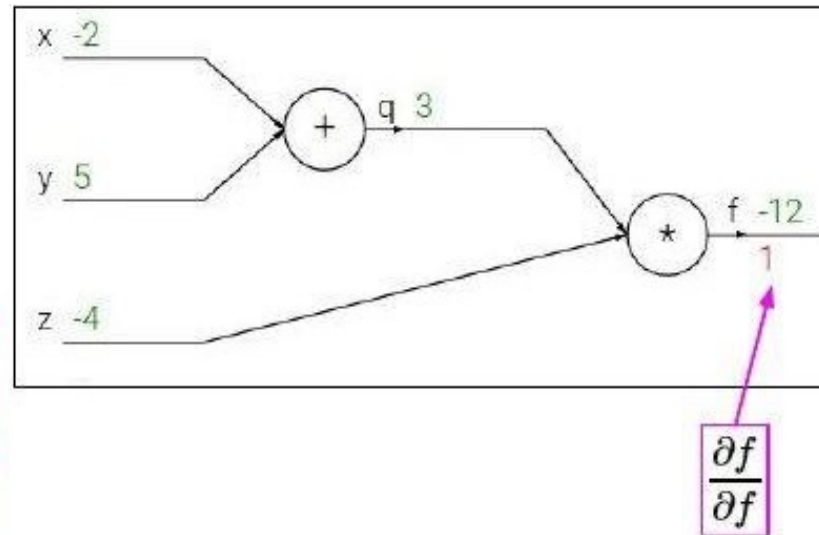
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

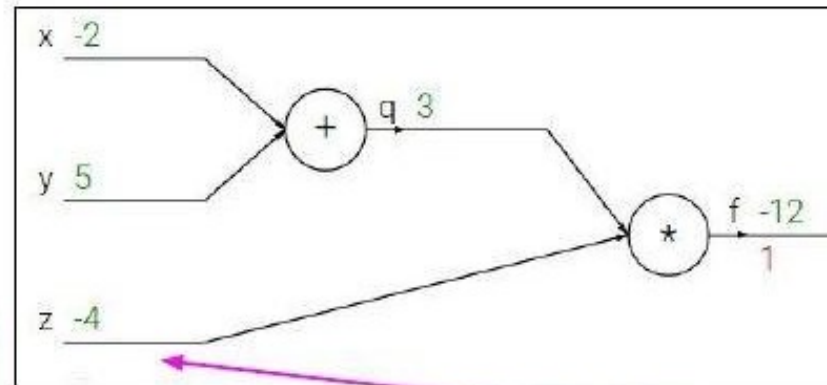
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

Computational Graph for Back Propagation

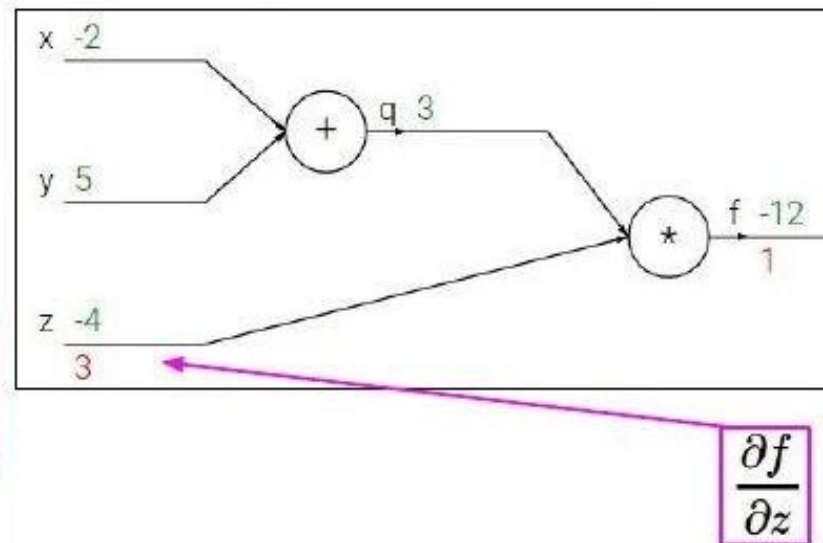
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation



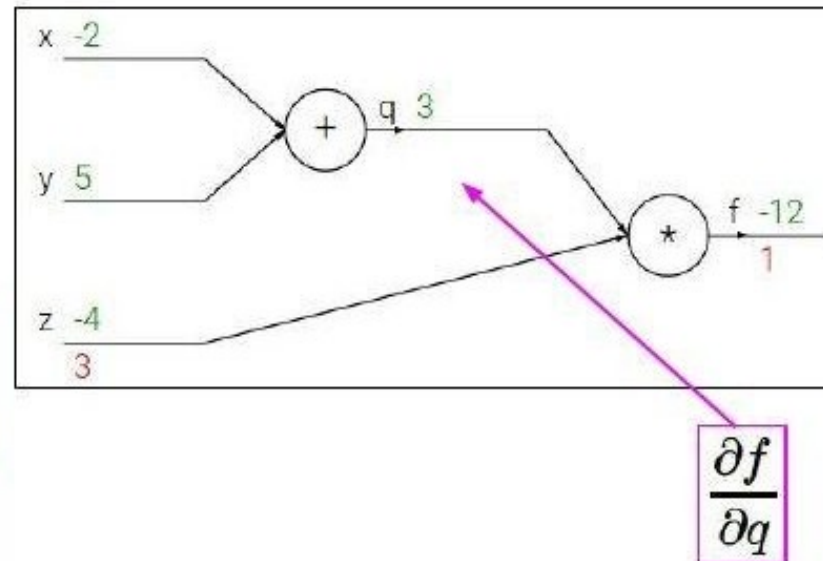
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

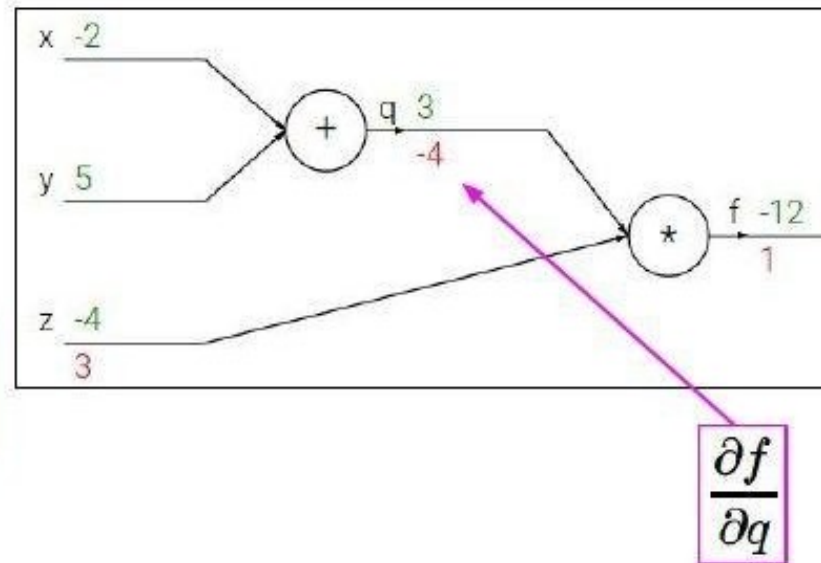
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Computational Graph for Back Propagation

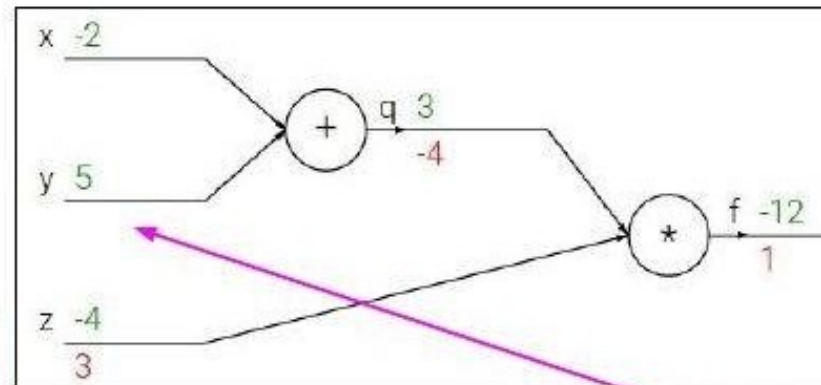
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient Local gradient

$$\frac{\partial f}{\partial y}$$

Computational Graph for Back Propagation

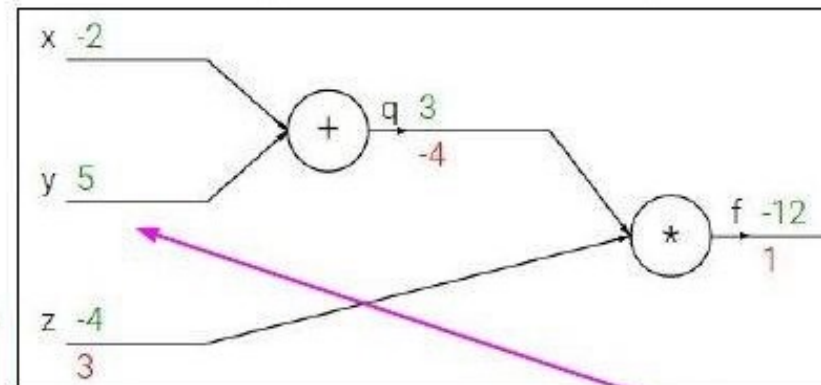
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream
gradient

Local
gradient

Computational Graph for Back Propagation

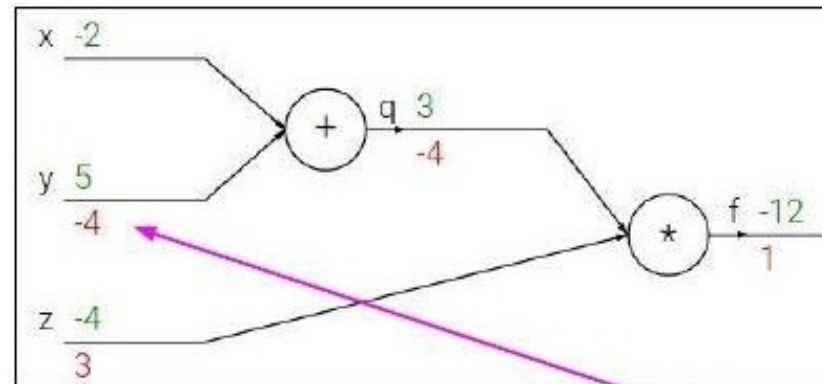
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream gradient Local gradient

$$\frac{\partial f}{\partial y}$$

Computational Graph for Back Propagation

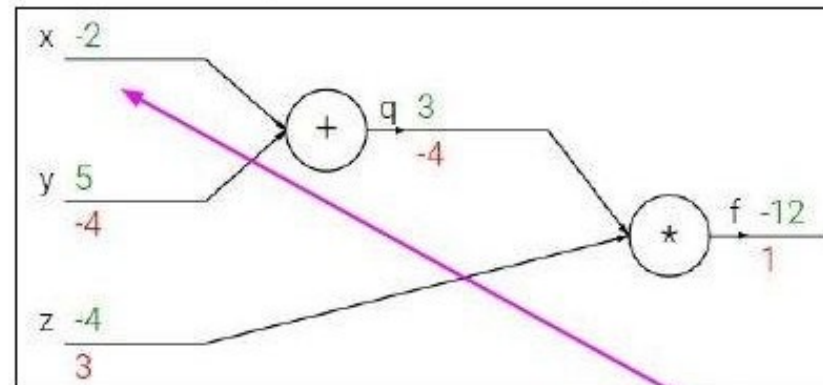
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

Computational Graph for Back Propagation

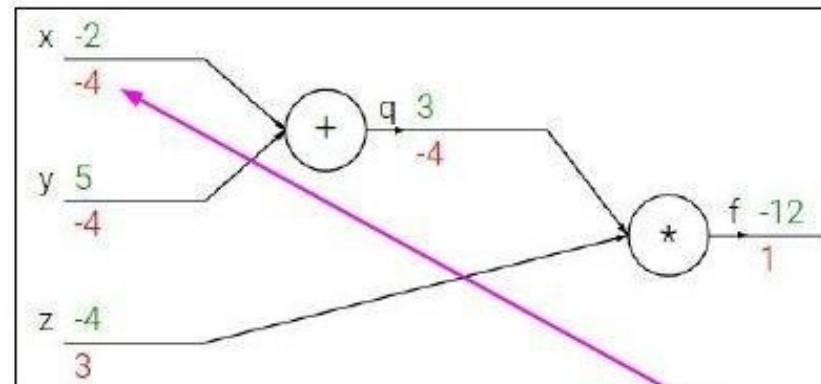
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

Upstream
gradient

Local
gradient

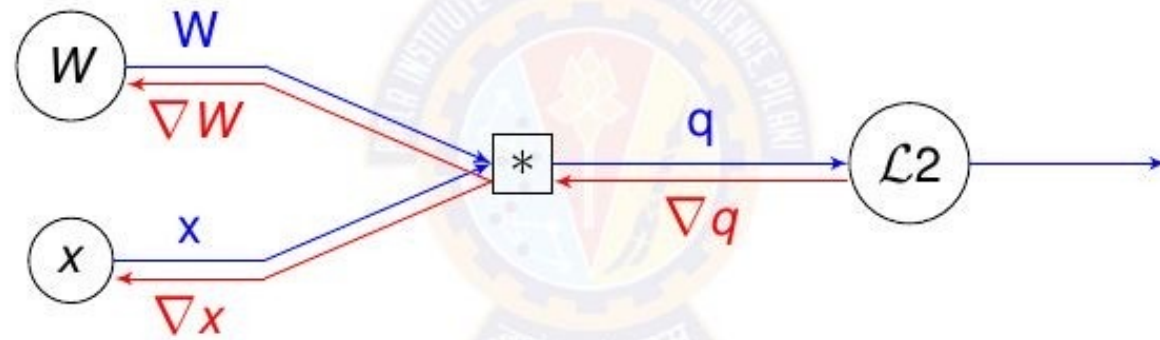
Example 2

Draw the computational graph for the equation

$$f(x, w) = ||W \cdot x||^2 = \sum_{i=1}^n (W \cdot x)_i^2$$

Using the computation graph show the computation of the gradients also. Assume that

$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$
$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$



$$W = \begin{bmatrix} 0.1 & 0.5 \\ -0.3 & 0.8 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$$

$$q = W \cdot x = \begin{bmatrix} 0.1 * 0.2 + 0.5 * 0.4 \\ -0.3 * 0.2 + 0.8 * 0.4 \end{bmatrix} = \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix}$$

$$L2 = \sum_{i=1} q_i^2 = 0.22^2 + 0.26^2 = 0.116$$

$$\nabla q = \nabla_q f = 2q$$

$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} 0.44 \\ 0.52 \end{bmatrix}$$

$$\nabla W = \nabla_W f = 2q \cdot x^\top$$

$$= 2 \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} \begin{bmatrix} 0.2 & 0.4 \end{bmatrix} = \begin{bmatrix} 0.088 & 0.176 \\ 0.105 & 0.208 \end{bmatrix}$$

$$\nabla x = \nabla_x f = 2W^\top \cdot q$$

$$= 2 \begin{bmatrix} 0.1 & -0.3 \\ 0.5 & 0.8 \end{bmatrix} \begin{bmatrix} 0.22 \\ 0.26 \end{bmatrix} = \begin{bmatrix} -0.112 \\ 0.636 \end{bmatrix}$$



Thank You All !